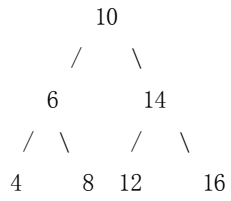
http://zhedahht.blog.163.com/blog/static/254111742007127104759245/

程序员面试题精选 100 题(01)－把二元查找树转变成排序的双向链表

题目：输入一棵二元查找树，将该二元查找树转换成一个排序的双向链表。要求不能创建任何新的结点，只调整指针的指向。

比如将二元查找树



转换成双向链表

4=6=8=10=12=14=16。

分析：本题是微软的面试题。很多与树相关的题目都是用递归的思路来解决，本题也不例外。下面我们用两种不同的递归思路来分析。

思路一：当我们到达某一结点准备调整以该结点为根结点的子树时，先调整其左子树将左子树转换成一个排好序的左子链表，再调整其右子树转换右子链表。最近链接左子链表的最右结点（左子树的最大结点）、当前结点和右子链表的最左结点（右子树的最小结点）。从树的根结点开始递归调整所有结点。

思路二：我们可以中序遍历整棵树。按照这个方式遍历树，比较小的结点先访问。如果我们每访问一个结点，假设之前访问过的结点已经调整成一个排序双向链表，我们再把调整当前结点的指针将其链接到链表的末尾。当所有结点都访问过之后，整棵树也就转换成一个排序双向链表了。

参考代码：

首先我们定义二元查找树结点的数据结构如下：

```
struct BSTreeNode // a node in the binary search tree
{
 int         m_nValue; // value of node
BSTreeNode  *m_pLeft;  // left child of node
BSTreeNode  *m_pRight; // right child of node
};
```

思路一对应的代码：

```
//////////////////////////////////////////////////////////////////////
//
// Covert a sub binary-search-tree into a sorted double-linked list
// Input: pNode - the head of the sub tree
//       asRight - whether pNode is the right child of its parent
// Output: if asRight is true, return the least node in the sub-tree
//        else return the greatest node in the sub-tree
//////////////////////////////////////////////////////////////////////
//
BSTreeNode* ConvertNode(BSTreeNode* pNode, bool asRight)
{
    if(!pNode)
        return NULL;

    BSTreeNode *pLeft = NULL;
    BSTreeNode *pRight = NULL;

    // Convert the left sub-tree
    if(pNode->m_pLeft)
        pLeft = ConvertNode(pNode->m_pLeft, false);

    // Connect the greatest node in the left sub-tree to the current node
    if(pLeft)
    {
        pLeft->m_pRight = pNode;
        pNode->m_pLeft = pLeft;
    }

    // Convert the right sub-tree
    if(pNode->m_pRight)
        pRight = ConvertNode(pNode->m_pRight, true);

    // Connect the least node in the right sub-tree to the current node
    if(pRight)
    {
        pNode->m_pRight = pRight;
        pRight->m_pLeft = pNode;
    }

    BSTreeNode *pTemp = pNode;

    // If the current node is the right child of its parent,
    // return the least node in the tree whose root is the current node
```

```
    if(asRight)
    {
        while(pTemp->m_pLeft)
            pTemp = pTemp->m_pLeft;
    }
    // If the current node is the left child of its parent,
    // return the greatest node in the tree whose root is the current node
    else
    {
        while(pTemp->m_pRight)
            pTemp = pTemp->m_pRight;
    }

    return pTemp;
}

//////////////////////////////////////////////////////////////////////
//
// Covert a binary search tree into a sorted double-linked list
// Input: the head of tree
// Output: the head of sorted double-linked list
//////////////////////////////////////////////////////////////////////
//
BSTreeNode* Convert(BSTreeNode* pHeadOfTree)
{
    // As we want to return the head of the sorted double-linked list,
    // we set the second parameter to be true
    return ConvertNode(pHeadOfTree, true);
}
```

思路二对应的代码：

```
//////////////////////////////////////////////////////////////////////
//
// Covert a sub binary-search-tree into a sorted double-linked list
// Input: pNode -          the head of the sub tree
//       pLastNodeInList - the tail of the double-linked list
//////////////////////////////////////////////////////////////////////
//
void ConvertNode(BSTreeNode* pNode, BSTreeNode*& pLastNodeInList)
{
    if(pNode == NULL)
        return;

    BSTreeNode *pCurrent = pNode;
```

```
    // Convert the left sub-tree
    if (pCurrent->m_pLeft != NULL)
        ConvertNode(pCurrent->m_pLeft, pLastNodeInList);

    // Put the current node into the double-linked list
    pCurrent->m_pLeft = pLastNodeInList;
    if(pLastNodeInList != NULL)
        pLastNodeInList->m_pRight = pCurrent;

    pLastNodeInList = pCurrent;

    // Convert the right sub-tree
    if (pCurrent->m_pRight != NULL)
        ConvertNode(pCurrent->m_pRight, pLastNodeInList);
}

//////////////////////////////////////////////////////////////////////
//
// Covert a binary search tree into a sorted double-linked list
// Input: pHeadOfTree - the head of tree
// Output: the head of sorted double-linked list
//////////////////////////////////////////////////////////////////////
//
BSTreeNode* Convert_Solution1(BSTreeNode* pHeadOfTree)
{
    BSTreeNode *pLastNodeInList = NULL;
    ConvertNode(pHeadOfTree, pLastNodeInList);

    // Get the head of the double-linked list
    BSTreeNode *pHeadOfList = pLastNodeInList;
    while(pHeadOfList && pHeadOfList->m_pLeft)
        pHeadOfList = pHeadOfList->m_pLeft;

    return pHeadOfList;
}
```

程序员面试题精选 100 题(02)－设计包含 min 函数的栈

题目：定义栈的数据结构，要求添加一个 min 函数，能够得到栈的最小元素。要求函数 min、push 以及 pop 的时间复杂度都是 O(1)。

分析：这是去年 google 的一道面试题。

我看到这道题目时，第一反应就是每次 push 一个新元素时，将栈里所有逆序元素排序。这样栈顶元素将是最小元素。但由于不能保证最后 push 进栈的元素最先出栈，这种思路设计的数据结构已经不是一个栈了。

在栈里添加一个成员变量存放最小元素（或最小元素的位置）。每次 push 一个新元素进栈的时候，如果该元素比当前的最小元素还要小，则更新最小元素。

乍一看这样思路挺好的。但仔细一想，该思路存在一个重要的问题：如果当前最小元素被 pop 出去，如何才能得到下一个最小元素？

因此仅仅只添加一个成员变量存放最小元素（或最小元素的位置）是不够的。我们需要一个辅助栈。每次 push 一个新元素的时候，同时将最小元素（或最小元素的位置。考虑到栈元素的类型可能是复杂的数据结构，用最小元素的位置将能减少空间消耗）push 到辅助栈中；每次 pop 一个元素出栈的时候，同时 pop 辅助栈。

参考代码：

```
#include <deque>
#include <assert.h>

template <typename T> class CStackWithMin
{
public:
    CStackWithMin(void) {}
    virtual ~CStackWithMin(void) {}

    T& top(void);
    const T& top(void) const;

    void push(const T& value);
    void pop(void);

    const T& min(void) const;

private:
    T>m_data;// theelements of stack
    size_t>m_minIndex;// the indicesof minimum elements
};

// get the last element of mutable stack
```

```
template <typename T> T& CStackWithMin<T>::top()
{
    return m_data.back();
}

// get the last element of non-mutable stack
template <typename T> const T& CStackWithMin<T>::top() const
{
    return m_data.back();
}

// insert an elment at the end of stack
template <typename T> void CStackWithMin<T>::push(const T& value)
{
    // append the data into the end of m_data
    m_data.push_back(value);

    // set the index of minimum elment in m_data at the end of m_minIndex
    if(m_minIndex.size() == 0)
        m_minIndex.push_back(0);
    else
    {
        if(value < m_data[m_minIndex.back()])
            m_minIndex.push_back(m_data.size() - 1);
        else
            m_minIndex.push_back(m_minIndex.back());
    }
}

// erease the element at the end of stack
template <typename T> void CStackWithMin<T>::pop()
{
    // pop m_data
    m_data.pop_back();

    // pop m_minIndex
    m_minIndex.pop_back();
}

// get the minimum element of stack
template <typename T> const T& CStackWithMin<T>::min() const
{
    assert(m_data.size() > 0);
    assert(m_minIndex.size() > 0);
```

```
    return m_data[m_minIndex.back()];
}
```

举个例子演示上述代码的运行过程：

步骤	数据栈	辅助栈	最小值
1.push 3	3	0	3
2.push 4	3,4	0,0	3
3.push 2	3,4,2	0,0,2	2
4.push 1	3,4,2,1	0,0,2,3	1
5.pop	3,4,2	0,0,2	2
6.pop	3,4	0,0	3
7.push 0	3,4,0	0,0,2	0

讨论：如果思路正确，编写上述代码不是一件很难的事情。但如果能注意一些细节无疑能在面试中加分。比如我在上面的代码中做了如下的工作：

- 用模板类实现。如果别人的元素类型只是 int 类型，模板将能给面试官带来好印象；
- 两个版本的 top 函数。在很多类中，都需要提供 const 和非 const 版本的成员访问函数；
- min 函数中 assert。把代码写的尽量安全是每个软件公司对程序员的要求；
- 添加一些注释。注释既能提高代码的可读性，又能增加代码量，何乐而不为？

总之，在面试时如果时间允许，尽量把代码写的漂亮一些。说不定代码中的几个小亮点就能让自己轻松拿到心仪的 Offer。

PS：每当 push 进一个新元素，若比当前最小元素小，则将它进栈，并将它的 index 进最小辅助栈；若大于当前最小元素，则将它进栈，并将当前最小元素 index 进最小辅助栈（可以重复进栈多次）！

程序员面试题精选 100 题(03)－求子数组的最大和

题目：输入一个整形数组，数组里有正数也有负数。数组中连续的一个或多个整数组成一个子数组，每个子数组都有一个和。求所有子数组的和的最大值。要求时间复杂度为 O(n)。

例如输入的数组为 1, -2, 3, 10, -4, 7, 2, -5，和最大的子数组为 3, 10, -4, 7, 2，因此输出为该子数组的和 18。

分析：本题最初为 2005 年浙江大学计算机系的考研题的最后一道程序设计题，在 2006 年里包括 google 在内的很多知名公司都把本题当作面试题。由于本题在网络中广为流传，本题也顺利成为 2006 年程序员面试题中经典中的经典。

如果不考虑时间复杂度，我们可以枚举出所有子数组并求出他们的和。不过非常遗憾的是，由于长度为 n 的数组有 $O(n^2)$个子数组；而且求一个长度为 n 的数组的和的时间复杂度为 O(n)。因此这种思路的时间是 $O(n^3)$。

很容易理解，当我们加上一个正数时，和会增加；当我们加上一个负数时，和会减少。如果当前得到的和是个负数，那么这个和在接下来的累加中应该抛弃并重新清零，不然的话这个负数将会减少接下来的和。基于这样的思路，我们可以写出如下代码。

参考代码：

```
//////////////////////////////////////////////////////////////////////
////////
// Find the greatest sum of all sub-arrays
// Return value: if the input is valid, return true, otherwise return false
//////////////////////////////////////////////////////////////////////
////////
bool FindGreatestSumOfSubArray
(
    int *pData,          // an array
    unsigned int nLength, // the length of array
    int &nGreatestSum     // the greatest sum of all sub-arrays
)
{
    // if the input is invalid, return false
    if((pData == NULL) || (nLength == 0))
        return false;

    int nCurSum = nGreatestSum = 0;
    for(unsigned int i = 0; i < nLength; ++i)
    {
        nCurSum += pData[i];

        // if the current sum is negative, discard it
        if(nCurSum < 0)
            nCurSum = 0;

        // if a greater sum is found, update the greatest sum
        if(nCurSum > nGreatestSum)
            nGreatestSum = nCurSum;

    }
```

```
    // if all data are negative, find the greatest element in the array
    if(nGreatestSum == 0)
    {
        nGreatestSum = pData[0];
        for(unsigned int i = 1; i < nLength; ++i)
        {
            if(pData[i] > nGreatestSum)
                nGreatestSum = pData[i];
        }
    }

    return true;
}
```

讨论：上述代码中有两点值得和大家讨论一下：

- 函数的返回值不是子数组和的最大值，而是一个判断输入是否有效的标志。如果函数返回值的是子数组和的最大值，那么当输入一个空指针是应该返回什么呢？返回 0？那这个函数的用户怎么区分输入无效和子数组和的最大值刚好是 0 这两中情况呢？基于这个考虑，本人认为把子数组和的最大值以引用的方式放到参数列表中，同时让函数返回一个函数是否正常执行的标志。

- 输入有一类特殊情况需要特殊处理。当输入数组中所有整数都是负数时，子数组和的最大值就是数组中的最大元素。

《编程珠机》第八章，8.4 扫描算法。

采用类似分治算法的道理：前 i 个元素中，最大综合子数组要么在 i-1 个元素中(maxsofar)，要么截止到位置 i(maxendinghere)。

程序员面试题精选 100 题(04)－在二元树中找出和为某一值的所有路径

题目：输入一个整数和一棵二元树。从树的根结点开始往下访问一直到叶结点所经过的所有结点形成一条路径。打印出和与输入整数相等的所有路径。

例如输入整数 22 和如下二元树

```
        10
       /  \
      5    12
     /  \
    4    7
```

则打印出两条路径：10, 12 和 10, 5, 7。

二元树结点的数据结构定义为：

```
struct BinaryTreeNode // a node in the binary tree
{
    int            m_nValue; // value of node
    BinaryTreeNode  *m_pLeft;  // left child of node
    BinaryTreeNode  *m_pRight; // right child of node
};
```

分析：这是百度的一道笔试题，考查对树这种基本数据结构以及递归函数的理解。

当访问到某一结点时，把该结点添加到路径上，并累加当前结点的值。如果当前结点为叶结点并且当前路径的和刚好等于输入的整数，则当前的路径符合要求，我们把它打印出来。如果当前结点不是叶结点，则继续访问它的子结点。当前结点访问结束后，递归函数将自动回到父结点。因此我们在函数退出之前要在路径上删除当前结点并减去当前结点的值，以确保返回父结点时路径刚好是根结点到父结点的路径。我们不难看出保存路径的数据结构实际上是一个栈结构，因为路径要与递归调用状态一致，而递归调用本质就是一个压栈和出栈的过程。

参考代码：

```
///////////////////////////////////////////////////////////////////////
//
// Find paths whose sum equal to expected sum
///////////////////////////////////////////////////////////////////////
//
void FindPath
(
    BinaryTreeNode*   pTreeNode,    // a node of binary tree
    int             expectedSum,  // the expected sum
    std::vector<int>&path,        // a pathfrom root to current node
    int&            currentSum    // the sum of path
)
{
    if(!pTreeNode)
        return;

    currentSum += pTreeNode->m_nValue;
```

```
    path.push_back(pTreeNode->m_nValue);

    // if the node is a leaf, and the sum is same as pre-defined,
    // the path is what we want. print the path
    bool isLeaf = (!pTreeNode->m_pLeft && !pTreeNode->m_pRight);
    if(currentSum == expectedSum && isLeaf)
    {
std::vector<int>::iterator iter =path.begin();
for(; iter != path.end(); ++ iter)
           std::cout<<*iter<<'\t';
        std::cout<<std::endl;
    }

    // if the node is not a leaf, goto its children
    if(pTreeNode->m_pLeft)
        FindPath(pTreeNode->m_pLeft, expectedSum, path, currentSum);
    if(pTreeNode->m_pRight)
        FindPath(pTreeNode->m_pRight, expectedSum, path, currentSum);

    // when we finish visiting a node and return to its parent node,
    // we should delete this node from the path and
    // minus the node's value from the current sum
    currentSum -= pTreeNode->m_nValue;
    path.pop_back();
}
```

程序员面试题精选 100 题(05)－查找最小的 k 个元素

题目：输入 n 个整数，输出其中最小的 k 个。

例如输入 1，2，3，4，5，6，7 和 8 这 8 个数字，则最小的 4 个数字为 1，2，3 和 4。

分析：这道题最简单的思路莫过于把输入的 n 个整数排序，这样排在最前面的 k 个数就是最小的 k 个数。只是这种思路的时间复杂度为 O(n*log*n)。我们试着寻找更快的解决思路。

我们可以开辟一个长度为 k 的数组。每次从输入的 n 个整数中读入一个数。如果数组中已经插入的元素少于 k 个，则将读入的整数直接放到数组中。否则长度为 k 的数组已经满了，不能再往数组里插入元素，只能替换了。如果读入的这个整数比数组中已有 k 个整数的最大值要小，则用读入的这个整数替换这个最大值；如果读入的整数比数组中已有 k 个整数的最大值还要大，则读入的这个整数不可能是最小的 k 个整数

之一，抛弃这个整数。这种思路相当于只要排序 k 个整数，因此时间复杂可以降到 O(n+n*log*k)。通常情况下 k 要远小于 n，所以这种办法要优于前面的思路。

这是我能够想出来的最快的解决方案。不过从给面试官留下更好印象的角度出发，我们可以进一步把代码写得更漂亮一些。从上面的分析，当长度为 k 的数组已经满了之后，如果需要替换，每次替换的都是数组中的最大值。在常用的数据结构中，能够在 O(1)时间里得到最大值的数据结构为最大堆。因此我们可以用堆（heap）来代替数组。

另外，自己重头开始写一个最大堆需要一定量的代码。我们现在不需要重新去发明车轮，因为前人早就发明出来了。同样，STL 中的 set 和 multiset 为我们做了很好的堆的实现，我们可以拿过来用。既偷了懒，又给面试官留下熟悉 STL 的好印象，何乐而不为之？

参考代码：

```
#include <set>
#include <vector>
#include <iostream>

using namespace std;

typedef multiset<int, greater<int> >  IntHeap;

//////////////////////////////////////////////////////////////////////
//
// find k least numbers in a vector
//////////////////////////////////////////////////////////////////////
//
void FindKLeastNumbers
(
    const vector<int>& data,              // a vector of data
    IntHeap& leastNumbers,                // k least numbers, output
    unsigned int k
)
{
    leastNumbers.clear();

    if(k == 0 || data.size() < k)
        return;

    vector<int>::const_iterator iter = data.begin();
    for(; iter != data.end(); ++ iter)
    {
        // if less than k numbers was inserted into leastNumbers
        if((leastNumbers.size()) < k)
            leastNumbers.insert(*iter);
```

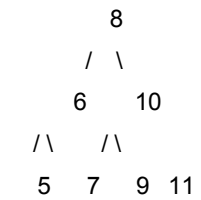
```
        // leastNumbers contains k numbers and it's full now
        else
        {
            // first number in leastNumbers is the greatest one
            IntHeap::iterator iterFirst = leastNumbers.begin();

            // if is less than the previous greatest number
            if(*iter < *(leastNumbers.begin()))
            {
                // replace the previous greatest number
                leastNumbers.erase(iterFirst);
                leastNumbers.insert(*iter);
            }
        }
    }
}
```

程序员面试题精选 100 题(06)－判断整数序列是不是二元查找树的后序遍历结果

题目：输入一个整数数组，判断该数组是不是某二元查找树的后序遍历的结果。如果是返回 true，否则返回 false。

例如输入 5、7、6、9、11、10、8，由于这一整数序列是如下树的后序遍历结果：



因此返回 true。

如果输入 7、4、6、5，没有哪棵树的后序遍历的结果是这个序列，因此返回 false。

分析：这是一道 trilogy 的笔试题，主要考查对二元查找树的理解。

在后续遍历得到的序列中，最后一个元素为树的根结点。从头开始扫描这个序列，比根结点小的元素都应该位于序列的左半部分；从第一个大于跟结点开始到跟结点前面的一个元素为止，所有元素都应该大于跟

结点，因为这部分元素对应的是树的右子树。根据这样的划分，把序列划分为左右两部分，我们递归地确认序列的左、右两部分是不是都是二元查找树。

参考代码：

```
using namespace std;

////////////////////////////////////////////////////////////////////////
//
// Verify whether a squence of integers are the post order traversal
// of a binary search tree (BST)
// Input: squence - the squence of integers
//        length  - the length of squence
// Return: return ture if the squence is traversal result of a BST,
//         otherwise, return false
////////////////////////////////////////////////////////////////////////
//
bool verifySquenceOfBST(int squence[], int length)
{
    if(squence == NULL || length <= 0)
        return false;

    // root of a BST is at the end of post order traversal squence
    int root = squence[length - 1];

    // the nodes in left sub-tree are less than the root
    int i = 0;
    for(; i < length - 1; ++ i)
    {
        if(squence[i] > root)
            break;
    }

    // the nodes in the right sub-tree are greater than the root
    int j = i;
    for(; j < length - 1; ++ j)
    {
        if(squence[j] < root)
            return false;
    }

    // verify whether the left sub-tree is a BST
    bool left = true;
    if(i > 0)
        left = verifySquenceOfBST(squence, i);
```

```
    // verify whether the right sub-tree is a BST
    bool right = true;
    if(i < length - 1)
        right = verifySquenceOfBST(squence + i, length - i - 1);

    return (left && right);
}
```

程序员面试题精选 100 题(07)－翻转句子中单词的顺序

题目：输入一个英文句子，翻转句子中单词的顺序，但单词内字符的顺序不变。句子中单词以空格符隔开。为简单起见，标点符号和普通字母一样处理。

例如输入“I am a student.”，则输出“student. a am I”。

分析：由于编写字符串相关代码能够反映程序员的编程能力和编程习惯，与字符串相关的问题一直是程序员笔试、面试题的热门题目。本题也曾多次受到包括微软在内的大量公司的青睐。

由于本题需要翻转句子，我们先颠倒句子中的所有字符。这时，不但翻转了句子中单词的顺序，而且单词内字符也被翻转了。我们再颠倒每个单词内的字符。由于单词内的字符被翻转两次，因此顺序仍然和输入时的顺序保持一致。

还是以上面的输入为例子。翻转“I am a student.”中所有字符得到“.tneduts a ma I”，再翻转每个单词中字符的顺序得到“students. a am I”，正是符合要求的输出。

参考代码：

```
//////////////////////////////////////////////////////////////////////
//
// Reverse a string between two pointers
// Input: pBegin - the begin pointer in a string
//        pEnd   - the end pointer in a string
//////////////////////////////////////////////////////////////////////
//
void Reverse(char *pBegin, char *pEnd)
{
    if(pBegin == NULL || pEnd == NULL)
        return;

    while(pBegin < pEnd)
```

```
    {
        char temp = *pBegin;
        *pBegin = *pEnd;
        *pEnd = temp;

        pBegin ++, pEnd --;
    }
}

//////////////////////////////////////////////////////////////////////
//
// Reverse the word order in a sentence, but maintain the character
// order inside a word
// Input: pData - the sentence to be reversed
//////////////////////////////////////////////////////////////////////
//
char* ReverseSentence(char *pData)
{
    if(pData == NULL)
        return NULL;

    char *pBegin = pData;
    char *pEnd = pData;

    while(*pEnd != '\0')
        pEnd ++;
    pEnd--;

    // Reverse the whole sentence
    Reverse(pBegin, pEnd);

    // Reverse every word in the sentence
    pBegin = pEnd = pData;
    while(*pBegin != '\0')
    {
        if(*pBegin == ' ')
        {
            pBegin ++;
            pEnd ++;
            continue;
        }
        // A word is between with pBegin and pEnd, reverse it
        else if(*pEnd == ' ' || *pEnd == '\0')
        {
```

```
            Reverse(pBegin, --pEnd);
            pBegin = ++pEnd;
        }
        else
        {
            pEnd ++;
        }
    }

    return pData;
}
```

程序员面试题精选 100 题(08)－求 1+2+...+n

题目：求 1+2+...+n，要求不能使用乘除法、for、while、if、else、switch、case 等关键字以及条件判断语句（A?B:C）。

分析：这道题没有多少实际意义，因为在软件开发中不会有这么变态的限制。但这道题却能有效地考查发散思维能力，而发散思维能力能反映出对编程相关技术理解的深刻程度。

通常求 1+2+...+n 除了用公式 n(n+1)/2 之外，无外乎循环和递归两种思路。由于已经明确限制 for 和 while 的使用，循环已经不能再用了。同样，递归函数也需要用 if 语句或者条件判断语句来判断是继续递归下去还是终止递归，但现在题目已经不允许使用这两种语句了。

我们仍然围绕循环做文章。循环只是让相同的代码执行 n 遍而已，我们完全可以不用 for 和 while 达到这个效果。比如定义一个类，我们 new 一含有 n 个这种类型元素的数组，那么该类的构造函数将确定会被调用 n 次。我们可以将需要执行的代码放到构造函数里。如下代码正是基于这个思路：

```
class Temp
{
public:
    Temp() { ++ N; Sum += N; }

    static void Reset() { N = 0; Sum = 0; }
    static int GetSum() { return Sum; }

private:
    static int N;
    static int Sum;
};
```

```
int Temp::N = 0;
int Temp::Sum = 0;

int solution1_Sum(int n)
{
      Temp::Reset();

    Temp *a = new Temp[n];
    delete []a;
    a = 0;

      return Temp::GetSum();
}
```

我们同样也可以围绕递归做文章。既然不能判断是不是应该终止递归，我们不妨定义两个函数。一个函数充当递归函数的角色，另一个函数处理终止递归的情况，我们需要做的就是在两个函数里二选一。从二选一我们很自然的想到布尔变量，比如 ture（1）的时候调用第一个函数，false（0）的时候调用第二个函数。那现在的问题是如和把数值变量 n 转换成布尔值。如果对 n 连续做两次反运算，即!!n，那么非零的 n 转换为 true，0 转换为 false。有了上述分析，我们再来看下面的代码：

```
class A;
A* Array[2];

class A
{
public:
    virtual int Sum (int n) { return 0; }
};

class B: public A
{
public:
    virtual int Sum (int n) { return Array[!!n]->Sum(n-1)+n; }
};

int solution2_Sum(int n)
{
    A a;
    B b;
    Array[0] = &a;
    Array[1] = &b;

    int value = Array[1]->Sum(n);
```

```
    return value;
}
```

这种方法是用虚函数来实现函数的选择。当 n 不为零时，执行函数 B::Sum；当 n 为 0 时，执行 A::Sum。我们也可以直接用函数指针数组，这样可能还更直接一些：

```
typedef int (*fun)(int);

int solution3_f1(int i)
{
    return 0;
}

int solution3_f2(int i)
{
    fun f[2]={solution3_f1, solution3_f2};
    return i+f[!!i](i-1);
}
```

另外我们还可以让编译器帮我们来完成类似于递归的运算，比如如下代码：

```
template <int n> struct solution4_Sum
{
    enum Value { N = solution4_Sum<n - 1>::N + n};
};

template <> struct solution4_Sum<1>
{
    enum Value { N = 1};
};
```

solution4_Sum<100>::N 就是 1+2+...+100 的结果。当编译器看到 solution4_Sum<100>时，就是为模板类 solution4_Sum 以参数 100 生成该类型的代码。但以 100 为参数的类型需要得到以 99 为参数的类型，因为 solution4_Sum<100>::N=solution4_Sum<99>::N+100。这个过程会递归一直到参数为 1 的类型，由于该类型已经显式定义，编译器无需生成，递归编译到此结束。由于这个过程是在编译过程中完成的，因此要求输入 n 必须是在编译期间就能确定，不能动态输入。这是该方法最大的缺点。而且编译器对递归编译代码的递归深度是有限制的，也就是要求 n 不能太大。

大家还有更多、更巧妙的思路吗？欢迎讨论^_^

PS:递归解决

```
int func(int n)
{
     int i=1;
```

```
    (n>1)&&(i=func(n-1)+n);
    return i;
}
```

程序员面试题精选 100 题(09)－查找链表中倒数第 k 个结点

题目：输入一个单向链表，输出该链表中倒数第 k 个结点。链表的倒数第 0 个结点为链表的尾指针。链表结点定义如下：

```
struct ListNode
{
    int      m_nKey;
    ListNode* m_pNext;
};
```

分析：为了得到倒数第 k 个结点，很自然的想法是先走到链表的尾端，再从尾端回溯 k 步。可是输入的是单向链表，只有从前往后的指针而没有从后往前的指针。因此我们需要打开我们的思路。

既然不能从尾结点开始遍历这个链表，我们还是把思路回到头结点上来。假设整个链表有 n 个结点，那么倒数第 k 个结点是从头结点开始的第 n-k-1 个结点（从 0 开始计数）。如果我们能够得到链表中结点的个数 n，那我们只要从头结点开始往后走 n-k-1 步就可以了。如何得到结点数 n？这个不难，只需要从头开始遍历链表，每经过一个结点，计数器加一就行了。

这种思路的时间复杂度是 O(n)，但需要遍历链表两次。第一次得到链表中结点个数 n，第二次得到从头结点开始的第 n-k-1 个结点即倒数第 k 个结点。

如果链表的结点数不多，这是一种很好的方法。但如果输入的链表的结点个数很多，有可能不能一次性把整个链表都从硬盘读入物理内存，那么遍历两遍意味着一个结点需要两次从硬盘读入到物理内存。我们知道把数据从硬盘读入到内存是非常耗时间的操作。我们能不能把链表遍历的次数减少到 1？如果可以，将能有效地提高代码执行的时间效率。

如果我们在遍历时维持两个指针，第一个指针从链表的头指针开始遍历，在第 k-1 步之前，第二个指针保持不动；在第 k-1 步开始，第二个指针也开始从链表的头指针开始遍历。由于两个指针的距离保持在 k-1，当第一个（走在前面的）指针到达链表的尾结点时，第二个指针（走在后面的）指针正好是倒数第 k 个结点。

这种思路只需要遍历链表一次。对于很长的链表，只需要把每个结点从硬盘导入到内存一次。因此这一方法的时间效率前面的方法要高。

思路一的参考代码：

```
////////////////////////////////////////////////////////////////////////
//
// Find the kth node from the tail of a list
// Input: pListHead - the head of list
//        k         - the distance to the tail
// Output: the kth node from the tail of a list
////////////////////////////////////////////////////////////////////////
//
ListNode* FindKthToTail_Solution1(ListNode* pListHead, unsigned int k)
{
    if(pListHead == NULL)
        return NULL;

    // count the nodes number in the list
    ListNode *pCur = pListHead;
    unsigned int nNum = 0;
    while(pCur->m_pNext != NULL)
    {
        pCur = pCur->m_pNext;
        nNum ++;
    }

    // if the number of nodes in the list is less than k
    // do nothing
    if(nNum < k)
        return NULL;

    // the kth node from the tail of a list
    // is the (n - k)th node from the head
    pCur = pListHead;
    for(unsigned int i = 0; i < nNum - k; ++ i)
        pCur = pCur->m_pNext;

    return pCur;
}
```

思路二的参考代码：

```
////////////////////////////////////////////////////////////////////////
//
// Find the kth node from the tail of a list
// Input: pListHead - the head of list
//        k         - the distance to the tail
// Output: the kth node from the tail of a list
////////////////////////////////////////////////////////////////////////
```

```
//
ListNode* FindKthToTail_Solution2(ListNode* pListHead, unsigned int k)
{
    if(pListHead == NULL)
        return NULL;

    ListNode *pAhead = pListHead;
    ListNode *pBehind = NULL;

    for(unsigned int i = 0; i < k; ++ i)
    {
        if(pAhead->m_pNext != NULL)
            pAhead = pAhead->m_pNext;
        else
        {
            // if the number of nodes in the list is less than k,
            // do nothing
            return NULL;
        }
    }

    pBehind = pListHead;

    // the distance between pAhead and pBehind is k
    // when pAhead arrives at the tail, p
    // Behind is at the kth node from the tail
    while(pAhead->m_pNext != NULL)
    {
        pAhead = pAhead->m_pNext;
        pBehind = pBehind->m_pNext;
    }

    return pBehind;
}
```

讨论：这道题的代码有大量的指针操作。在软件开发中，错误的指针操作是大部分问题的根源。因此每个公司都希望程序员在操作指针时有良好的习惯，比如使用指针之前判断是不是空指针。这些都是编程的细节，但如果这些细节把握得不好，很有可能就会和心仪的公司失之交臂。

另外，这两种思路对应的代码都含有循环。含有循环的代码经常出的问题是在循环结束条件的判断。是该用小于还是小于等于？是该用 k 还是该用 k-1？由于题目要求的是从 0 开始计数，而我们的习惯思维是从 1 开始计数，因此首先要想好这些边界条件再开始编写代码，再者要在编写完代码之后再用边界值、边界值减 1、边界值加 1 都运行一次（在纸上写代码就只能在心里运行了）。

扩展：和这道题类似的题目还有：输入一个单向链表。如果该链表的结点数为奇数，输出中间的结点；如果链表结点数为偶数，输出中间两个结点前面的一个。如果各位感兴趣，请自己分析并编写代码。

解扩展题的思路和思路二类似吧，也用到两个指针。节点数为奇数时，两个指针初始都指向头结点，然后一个每次跳两个节点，一个每次跳一个节点。当第一个指针到达尾端时后一个指针指向需要的节点。

节点数为偶数时情况基本一样，只是两个指针初始一个指向头结点，一个指向 1 号节点。后一个指针每次跳两个节点，前一个每次只跳一个节点。

程序员面试题精选 100 题(10)－在排序数组中查找和为给定值的两个数字

题目：输入一个已经按升序排序过的数组和一个数字，在数组中查找两个数，使得它们的和正好是输入的那个数字。要求时间复杂度是 O(n)。如果有多对数字的和等于输入的数字，输出任意一对即可。

例如输入数组 1、2、4、7、11、15 和数字 15。由于 4+11=15，因此输出 4 和 11。

分析：如果我们不考虑时间复杂度，最简单想法的莫过去先在数组中固定一个数字，再依次判断数组中剩下的 n-1 个数字与它的和是不是等于输入的数字。可惜这种思路需要的时间复杂度是 $O(n^2)$。

我们假设现在随便在数组中找到两个数。如果它们的和等于输入的数字，那太好了，我们找到了要找的两个数字；如果小于输入的数字呢？我们希望两个数字的和再大一点。由于数组已经排好序了，我们是不是可以把较小的数字的往后面移动一个数字？因为排在后面的数字要大一些，那么两个数字的和也要大一些，就有可能等于输入的数字了；同样，当两个数字的和大于输入的数字的时候，我们把较大的数字往前移动，因为排在数组前面的数字要小一些，它们的和就有可能等于输入的数字了。

我们把前面的思路整理一下：最初我们找到数组的第一个数字和最后一个数字。当两个数字的和大于输入的数字时，把较大的数字往前移动；当两个数字的和小于数字时，把较小的数字往后移动；当相等时，打完收工。这样扫描的顺序是从数组的两端向数组的中间扫描。

问题是这样的思路是不是正确的呢？这需要严格的数学证明。感兴趣的读者可以自行证明一下。

参考代码：

```
///////////////////////////////////////////////////////////////////////
//
// Find two numbers with a sum in a sorted array
// Output: ture is found such two numbers, otherwise false
///////////////////////////////////////////////////////////////////////
```

```
//
bool FindTwoNumbersWithSum
(
    int data[],          // a sorted array
    unsigned int length,  // the length of the sorted array
    int sum,             // the sum
    int& num1,           // the first number, output
    int& num2            // the second number, output
)
{

    bool found = false;
    if(length < 1)
        return found;

    int ahead = length - 1;
    int behind = 0;

    while(ahead > behind)
    {
        long long curSum = data[ahead] + data[behind];

        // if the sum of two numbers is equal to the input
        // we have found them
        if(curSum == sum)
        {
            num1 = data[behind];
            num2 = data[ahead];
            found = true;
            break;
        }
        // if the sum of two numbers is greater than the input
        // decrease the greater number
        else if(curSum > sum)
            ahead --;
        // if the sum of two numbers is less than the input
        // increase the less number
        else
            behind ++;
    }

    return found;
}
```

扩展：如果输入的数组是没有排序的，但知道里面数字的范围，其他条件不变，如和在 O(n)时间里找到这两个数字？

==========

扩展问题是不是先记数排序再用原来的方法？

如果是这样的话，设数字范围是 d，则时间复杂度应该是 O(max(d,n))

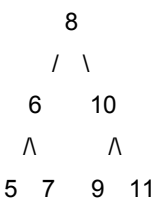
是这个思路。如果记最小值为 min，最大值为 max。新建一个长度为 max-min+1 的数组。初始化这个数组的每个元素为 0。扫描原数组每个元素 k，在新数组中下标为 k-min 的位置加 1，这样在 O(n)的时间内把原数组转换为一个排好序的数组。接下来的做法一样。

当然，时间复杂度标记为 O(max(d,n)) 更准确一些。谢谢指出。

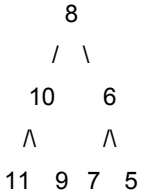
程序员面试题精选 100 题(11)－求二元查找树的镜像

题目：输入一颗二元查找树，将该树转换为它的镜像，即在转换后的二元查找树中，左子树的结点都大于右子树的结点。用递归和循环两种方法完成树的镜像转换。

例如输入：



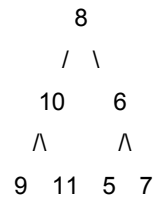
输出：



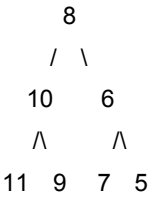
定义二元查找树的结点为：

```
struct BSTreeNode // a node in the binary search tree (BST)
{
    int         m_nValue; // value of node
    BSTreeNode  *m_pLeft;  // left child of node
    BSTreeNode  *m_pRight; // right child of node
};
```

分析：尽管我们可能一下子不能理解镜像是什么意思，但上面的例子给我们的直观感觉，就是交换结点的左右子树。我们试着在遍历例子中的二元查找树的同时来交换每个结点的左右子树。遍历时首先访问头结点 8，我们交换它的左右子树得到：



我们发现两个结点 6 和 10 的左右子树仍然是左结点的值小于右结点的值，我们再试着交换他们的左右子树，得到：



刚好就是要求的输出。

上面的分析印证了我们的直觉：在遍历二元查找树时每访问到一个结点，交换它的左右子树。这种思路用递归不难实现，将遍历二元查找树的代码稍作修改就可以了。参考代码如下：

```
//////////////////////////////////////////////////////////////////////
//
// Mirror a BST (swap the left right child of each node) recursively
// the head of BST in initial call
//////////////////////////////////////////////////////////////////////
//
void MirrorRecursively(BSTreeNode *pNode)
{
    if(!pNode)
        return;

    // swap the right and left child sub-tree
    BSTreeNode *pTemp = pNode->m_pLeft;
    pNode->m_pLeft = pNode->m_pRight;
```

```
    pNode->m_pRight = pTemp;

// mirror left child sub-tree if not null
    if(pNode->m_pLeft)
        MirrorRecursively(pNode->m_pLeft);

    // mirror right child sub-tree if not null
    if(pNode->m_pRight)
        MirrorRecursively(pNode->m_pRight);
}
```

由于递归的本质是编译器生成了一个函数调用的栈，因此用循环来完成同样任务时最简单的办法就是用一个辅助栈来模拟递归。首先我们把树的头结点放入栈中。在循环中，只要栈不为空，弹出栈的栈顶结点，交换它的左右子树。如果它有左子树，把它的左子树压入栈中；如果它有右子树，把它的右子树压入栈中。这样在下次循环中就能交换它儿子结点的左右子树了。参考代码如下：

```
//////////////////////////////////////////////////////////////////////
//
// Mirror a BST (swap the left right child of each node) Iteratively
// Input: pTreeHead: the head of BST
//////////////////////////////////////////////////////////////////////
//
void MirrorIteratively(BSTreeNode *pTreeHead)
{
    if(!pTreeHead)
        return;

    std::stack<BSTreeNode*>stackTreeNode;
    stackTreeNode.push(pTreeHead);

while(stackTreeNode.size())
    {
        BSTreeNode *pNode = stackTreeNode.top();
        stackTreeNode.pop();

        // swap the right and left child sub-tree
        BSTreeNode *pTemp = pNode->m_pLeft;
        pNode->m_pLeft = pNode->m_pRight;
        pNode->m_pRight = pTemp;

        // push left child sub-tree into stack if not null
        if(pNode->m_pLeft)
            stackTreeNode.push(pNode->m_pLeft);

        // push right child sub-tree into stack if not null
```

```
        if(pNode->m_pRight)
            stackTreeNode.push(pNode->m_pRight);
    }
}
```

程序员面试题精选 100 题(12)－从上往下遍历二元树

题目：输入一颗二元树，从上往下按层打印树的每个结点，同一层中按照从左往右的顺序打印。

例如输入

```
     8
    / \
   6   10
  /\    /\
 5  7  9  11
```

输出 8　6　10　5　7　9　11。

分析：这曾是微软的一道面试题。这道题实质上是要求遍历一棵二元树，只不过不是我们熟悉的前序、中序或者后序遍历。

我们从树的根结点开始分析。自然先应该打印根结点 8，同时为了下次能够打印 8 的两个子结点，我们应该在遍历到 8 时把子结点 6 和 10 保存到一个数据容器中。现在数据容器中就有两个元素 6 和 10 了。按照从左往右的要求，我们先取出 6 访问。打印 6 的同时要把 6 的两个子结点 5 和 7 放入数据容器中，此时数据容器中有三个元素 10、5 和 7。接下来我们应该从数据容器中取出结点 10 访问了。注意 10 比 5 和 7 先放入容器，此时又比 5 和 7 先取出，就是我们通常说的先入先出。因此不难看出这个数据容器的类型应该是个队列。

既然已经确定数据容器是一个队列，现在的问题变成怎么实现队列了。实际上我们无需自己动手实现一个，因为 STL 已经为我们实现了一个很好的 deque（两端都可以进出的队列），我们只需要拿过来用就可以了。

我们知道树是图的一种特殊退化形式。同时如果对图的深度优先遍历和广度优先遍历有比较深刻的理解，将不难看出这种遍历方式实际上是一种广度优先遍历。因此这道题的本质是在二元树上实现广度优先遍历。

参考代码：

```
#include <deque>
#include <iostream>
using namespace std;
```

```
struct BTreeNode // a node in the binary tree
{
    int        m_nValue; // value of node
    BTreeNode  *m_pLeft;  // left child of node
    BTreeNode  *m_pRight; // right child of node
};

////////////////////////////////////////////////////////////////////////
//
// Print a binary tree from top level to bottom level
// Input: pTreeRoot - the root of binary tree
////////////////////////////////////////////////////////////////////////
//
void PrintFromTopToBottom(BTreeNode *pTreeRoot)
{
    if(!pTreeRoot)
        return;

    // get a empty queue
    deque<BTreeNode *> dequeTreeNode;

    // insert the root at the tail of queue
    dequeTreeNode.push_back(pTreeRoot);

    while(dequeTreeNode.size())
    {
        // get a node from the head of queue
        BTreeNode *pNode = dequeTreeNode.front();
        dequeTreeNode.pop_front();

        // print the node
        cout << pNode->m_nValue << ' ';

        // print its left child sub-tree if it has
        if(pNode->m_pLeft)
            dequeTreeNode.push_back(pNode->m_pLeft);
        // print its right child sub-tree if it has
        if(pNode->m_pRight)
            dequeTreeNode.push_back(pNode->m_pRight);
    }
}
```

PS：层序二叉树，用队列实现！

程序员面试题精选 100 题(13)－第一个只出现一次的字符

题目：在一个字符串中找到第一个只出现一次的字符。如输入 abaccdeff，则输出 b。

分析：这道题是 2006 年 google 的一道笔试题。

看到这道题时，最直观的想法是从头开始扫描这个字符串中的每个字符。当访问到某字符时拿这个字符和后面的每个字符相比较，如果在后面没有发现重复的字符，则该字符就是只出现一次的字符。如果字符串有 n 个字符，每个字符可能与后面的 O(n)个字符相比较，因此这种思路时间复杂度是 $O(n^2)$。我们试着去找一个更快的方法。

由于题目与字符出现的次数相关，我们是不是可以统计每个字符在该字符串中出现的次数？要达到这个目的，我们需要一个数据容器来存放每个字符的出现次数。在这个数据容器中可以根据字符来查找它出现的次数，也就是说这个容器的作用是把一个字符映射成一个数字。在常用的数据容器中，哈希表正是这个用途。

哈希表是一种比较复杂的数据结构。由于比较复杂，STL 中没有实现哈希表，因此需要我们自己实现一个。但由于本题的特殊性，我们只需要一个非常简单的哈希表就能满足要求。由于字符（char）是一个长度为 8 的数据类型，因此总共有可能 256 种可能。于是我们创建一个长度为 256 的数组，每个字母根据其 ASCII 码值作为数组的下标对应数组的对应项，而数组中存储的是每个字符对应的次数。这样我们就创建了一个大小为 256，以字符 ASCII 码为键值的哈希表。

我们第一遍扫描这个数组时，每碰到一个字符，在哈希表中找到对应的项并把出现的次数增加一次。这样在进行第二次扫描时，就能直接从哈希表中得到每个字符出现的次数了。

参考代码如下：

```
//////////////////////////////////////////////////////////////////////
//
// Find the first char which appears only once in a string
// Input: pString - the string
// Output: the first not repeating char if the string has, otherwise 0
//////////////////////////////////////////////////////////////////////
//
char FirstNotRepeatingChar(char* pString)
{
    // invalid input
    if(!pString)
        return 0;

    // get a hash table, and initialize it
        constinttableSize =256;
unsignedinthashTable[tableSize];
```

```
for(unsignedinti = 0; i<tableSize; ++ i)
hashTable[i] = 0;

    // get the how many times each char appears in the string
    char* pHashKey = pString;
    while(*(pHashKey) != '\0')
        hashTable[*(pHashKey++)] ++;

    // find the first char which appears only once in a string
    pHashKey = pString;
    while(*pHashKey != '\0')
    {
        if(hashTable[*pHashKey] == 1)
            return *pHashKey;

        pHashKey++;
    }

    // if the string is empty
    // or every char in the string appears at least twice
    return 0;
}
```

程序员面试题精选 100 题(14)－圆圈中最后剩下的数字

题目：n 个数字（0,1,…,n-1）形成一个圆圈，从数字 0 开始，每次从这个圆圈中删除第 m 个数字（第一个为当前数字本身，第二个为当前数字的下一个数字）。当一个数字删除后，从被删除数字的下一个继续删除第 m 个数字。求出在这个圆圈中剩下的最后一个数字。

分析：既然题目有一个数字圆圈，很自然的想法是我们用一个数据结构来模拟这个圆圈。在常用的数据结构中，我们很容易想到用环形列表。我们可以创建一个总共有 m 个数字的环形列表，然后每次从这个列表中删除第 m 个元素。

在参考代码中，我们用 STL 中 std::list 来模拟这个环形列表。由于 list 并不是一个环形的结构，因此每次跌代器扫描到列表末尾的时候，要记得把跌代器移到列表的头部。这样就是按照一个圆圈的顺序来遍历这个列表了。

这种思路需要一个有 n 个结点的环形列表来模拟这个删除的过程，因此内存开销为 O(n)。而且这种方法每删除一个数字需要 m 步运算，总共有 n 个数字，因此总的时间复杂度是 O(mn)。当 m 和 n 都很大的时候，这种方法是很慢的。

接下来我们试着从数学上分析出一些规律。首先定义最初的 n 个数字（0,1,…,n-1）中最后剩下的数字是关于 n 和 m 的方程为 f(n,m)。

在这 n 个数字中，第一个被删除的数字是 m%n-1，为简单起见记为 k。那么删除 k 之后的剩下 n-1 的数字为 0,1,…,k-1,k+1,…,n-1，并且下一个开始计数的数字是 k+1。相当于在剩下的序列中，k+1 排到最前面，从而形成序列 k+1,…,n-1,0,…k-1。该序列最后剩下的数字也应该是关于 n 和 m 的函数。由于这个序列的规律和前面最初的序列不一样（最初的序列是从 0 开始的连续序列），因此该函数不同于前面函数，记为 f'(n-1,m)。最初序列最后剩下的数字 f(n,m)一定是剩下序列的最后剩下数字 f'(n-1,m)，所以 f(n,m)=f'(n-1,m)。

接下来我们把剩下的的这 n-1 个数字的序列 k+1,…,n-1,0,…k-1 作一个映射，映射的结果是形成一个从 0 到 n-2 的序列：

k+1 -> 0
k+2 -> 1
…
n-1 -> n-k-2
0 -> n-k-1
…
k-1 -> n-2

把映射定义为 p，则 p(x)= (x-k-1)%n，即如果映射前的数字是 x，则映射后的数字是(x-k-1)%n。对应的逆映射是 $p^{-1}(x)=(x+k+1)\%n$。

由于映射之后的序列和最初的序列有同样的形式，都是从 0 开始的连续序列，因此仍然可以用函数 f 来表示，记为 f(n-1,m)。根据我们的映射规则，映射之前的序列最后剩下的数字 $f'(n-1,m)= p^{-1}[f(n-1,m)]=[f(n-1,m)+k+1]\%n$。把 k=m%n-1 代入得到 f(n,m)=f'(n-1,m)=[f(n-1,m)+m]%n。

经过上面复杂的分析，我们终于找到一个递归的公式。要得到 n 个数字的序列的最后剩下的数字，只需要得到 n-1 个数字的序列的最后剩下的数字，并可以依此类推。当 n=1 时，也就是序列中开始只有一个数字 0，那么很显然最后剩下的数字就是 0。我们把这种关系表示为：

$$f(n,m)=\begin{cases}0 & n=1\\ [f(n-1,m)+m]\%n & n>1\end{cases}$$

尽管得到这个公式的分析过程非常复杂，但它用递归或者循环都很容易实现。最重要的是，这是一种时间复杂度为 O(n)，空间复杂度为 O(1)的方法，因此无论在时间上还是空间上都优于前面的思路。

思路一的参考代码：

```
//////////////////////////////////////////////////////////////////////
//
// n integers (0, 1, ... n - 1) form a circle. Remove the mth from
// the circle at every time. Find the last number remaining
// Input: n - the number of integers in the circle initially
```

```
//       m - remove the mth number at every time
// Output: the last number remaining when the input is valid,
//        otherwise -1
////////////////////////////////////////////////////////////////////
//
int LastRemaining_Solution1(unsigned int n, unsigned int m)
{
    // invalid input
    if(n < 1 || m < 1)
        return -1;

    unsigned int i = 0;

    // initiate a list with n integers (0, 1, ... n - 1)
    list<int> integers;
    for(i = 0; i < n; ++ i)
        integers.push_back(i);

    list<int>::iterator curinteger = integers.begin();
    while(integers.size() > 1)
    {
        // find the mth integer. Note that std::list is not a circle
        // so we should handle it manually
        for(int i = 1; i < m; ++ i)
        {
            curinteger ++;
            if(curinteger == integers.end())
                curinteger = integers.begin();
        }

        // remove the mth integer. Note that std::list is not a circle
        // so we should handle it manually
        list<int>::iterator nextinteger = ++ curinteger;
        if(nextinteger == integers.end())
            nextinteger = integers.begin();

        -- curinteger;
        integers.erase(curinteger);
        curinteger = nextinteger;
    }

    return *(curinteger);
}
```

思路二的参考代码：

```
//////////////////////////////////////////////////////////////////////
//
// n integers (0, 1, ... n - 1) form a circle. Remove the mth from
// the circle at every time. Find the last number remaining
// Input: n - the number of integers in the circle initially
//        m - remove the mth number at every time
// Output: the last number remaining when the input is valid,
//         otherwise -1
//////////////////////////////////////////////////////////////////////
//
int LastRemaining_Solution2(int n, unsigned int m)
{
    // invalid input
    if(n <= 0 || m < 0)
        return -1;

    // if there are only one integer in the circle initially,
    // of course the last remaining one is 0
    int lastinteger = 0;

    // find the last remaining one in the circle with n integers
    for (int i = 2; i <= n; i ++)
        lastinteger = (lastinteger + m) % i;

    return lastinteger;
}
```

如果对两种思路的时间复杂度感兴趣的读者可以把 n 和 m 的值设的稍微大一点，比如十万这个数量级的数字，运行的时候就能明显感觉出这两种思路写出来的代码时间效率大不一样。

程序员面试题精选 100 题(15)－含有指针成员的类的拷贝

题目：下面是一个数组类的声明与实现。请分析这个类有什么问题，并针对存在的问题提出几种解决方案。

```
template<typename T> class Array
{
public:
    Array(unsigned arraySize):data(0), size(arraySize)
```

```
    {
        if(size > 0)
            data = new T[size];
    }

    ~Array()
    {
        if(data) delete[] data;
    }

    void setValue(unsigned index, const T& value)
    {
        if(index < size)
            data[index] = value;
    }

    T getValue(unsigned index) const
    {
        if(index < size)
            return data[index];
        else
            return T();
    }

private:
    T* data;
    unsigned size;
};
```

分析：我们注意在类的内部封装了用来存储数组数据的指针。软件存在的大部分问题通常都可以归结指针的不正确处理。

这个类只提供了一个构造函数，而没有定义构造拷贝函数和重载拷贝运算符函数。当这个类的用户按照下面的方式声明并实例化该类的一个实例

```
Array A(10);
Array B(A);
```

或者按照下面的方式把该类的一个实例赋值给另外一个实例

```
Array A(10);
Array B(10);
B=A;
```

编译器将调用其自动生成的构造拷贝函数或者拷贝运算符的重载函数。在编译器生成的缺省的构造拷贝函数和拷贝运算符的重载函数，对指针实行的是按位拷贝，仅仅只是拷贝指针的地址，而不会拷贝指针的内容。因此在执行完前面的代码之后，A.data 和 B.data 指向的同一地址。当 A 或者 B 中任意一个结束其生命周期调用析构函数时，会删除 data。由于他们的 data 指向的是同一个地方，两个实例的 data 都被删除了。但另外一个实例并不知道它的 data 已经被删除了，当企图再次用它的 data 的时候，程序就会不可避免地崩溃。

由于问题出现的根源是调用了编译器生成的缺省构造拷贝函数和拷贝运算符的重载函数。一个最简单的办法就是禁止使用这两个函数。于是我们可以把这两个函数声明为私有函数，如果类的用户企图调用这两个函数，将不能通过编译。实现的代码如下：

```
private:
    Array(const Array& copy);
    const Array& operator = (const Array& copy);
```

最初的代码存在问题是因为不同实例的 data 指向的同一地址，删除一个实例的 data 会把另外一个实例的 data 也同时删除。因此我们还可以让构造拷贝函数或者拷贝运算符的重载函数拷贝的不只是地址，而是数据。由于我们重新存储了一份数据，这样一个实例删除的时候，对另外一个实例没有影响。这种思路我们称之为深度拷贝。实现的代码如下：

```
public:
    Array(const Array& copy):data(0), size(copy.size)
    {
        if(size > 0)
        {
            data = new T[size];
            for(int i = 0; i < size; ++ i)
                setValue(i, copy.getValue(i));
        }
    }

    const Array& operator = (const Array& copy)
    {
        if(this == &copy)
            return *this;

        if(data != NULL)
        {
            delete []data;
            data = NULL;
        }

        size = copy.size;
        if(size > 0)
        {
```

```
            data = new T[size];
            for(int i = 0; i < size; ++ i)
                setValue(i, copy.getValue(i));
        }
    }
```

为了防止有多个指针指向的数据被多次删除，我们还可以保存究竟有多少个指针指向该数据。只有当没有任何指针指向该数据的时候才可以被删除。这种思路通常被称之为引用计数技术。在构造函数中，引用计数初始化为 1；每当把这个实例赋值给其他实例或者以参数传给其他实例的构造拷贝函数的时候，引用计数加 1，因为这意味着又多了一个实例指向它的 data；每次需要调用析构函数或者需要把 data 赋值为其他数据的时候，引用计数要减 1，因为这意味着指向它的 data 的指针少了一个。当引用计数减少到 0 的时候，data 已经没有任何实例指向它了，这个时候就可以安全地删除。实现的代码如下：

```
public:
    Array(unsigned arraySize)
        :data(0), size(arraySize), count(new unsigned int)
    {
        *count = 1;
        if(size > 0)
            data = new T[size];
    }

    Array(const Array& copy)
        : size(copy.size), data(copy.data), count(copy.count)
    {
        ++ (*count);
    }

    ~Array()
    {
        Release();
    }

    const Array& operator = (const Array& copy)
    {
        if(data == copy.data)
            return *this;

        Release();

        data = copy.data;
        size = copy.size;
        count = copy.count;
        ++(*count);
    }
```

```
private:
    void Release()
    {
        --(*count);
        if(*count == 0)
        {
            if(data)
            {
                delete []data;
                data = NULL;
            }

            delete count;
            count = 0;
        }
    }

    unsigned int *count;
```

PS：拷贝构造函数和重载拷贝运算符函数，深复制，避免调用缺省（浅复制），两对象指向同一地址，删除一个时出现错误！

程序员面试题精选 100 题(16)－O(logn)求 Fibonacci 数列

题目：定义 Fibonacci 数列如下：

```
       /  0                    n=0
f(n)=      1                    n=1
       \  f(n-1)+f(n-2)          n=2
```

输入 n，用最快的方法求该数列的第 n 项。

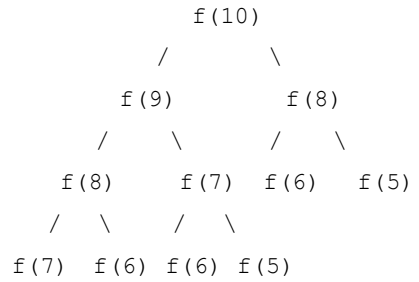
分析：在很多 C 语言教科书中讲到递归函数的时候，都会用 Fibonacci 作为例子。因此很多程序员对这道题的递归解法非常熟悉，看到题目就能写出如下的递归求解的代码。

```
//////////////////////////////////////////////////////////////////////
//
// Calculate the nth item of Fibonacci Series recursively
//////////////////////////////////////////////////////////////////////
//
```

```
long long Fibonacci_Solution1(unsigned int n)
{
    int result[2] = {0, 1};
    if(n < 2)
        return result[n];

    return Fibonacci_Solution1(n - 1) + Fibonacci_Solution1(n - 2);
}
```

但是，教科书上反复用这个题目来讲解递归函数，并不能说明递归解法最适合这道题目。我们以求解 f(10) 作为例子来分析递归求解的过程。要求得 f(10)，需要求得 f(9)和 f(8)。同样，要求得 f(9)，要先求得 f(8)和 f(7)……我们用树形结构来表示这种依赖关系



我们不难发现在这棵树中有很多结点会重复的，而且重复的结点数会随着 n 的增大而急剧增加。这意味这计算量会随着 n 的增大而急剧增大。事实上，用递归方法计算的时间复杂度是以 n 的指数的方式递增的。大家可以求 Fibonacci 的第 100 项试试，感受一下这样递归会慢到什么程度。在我的机器上，连续运行了一个多小时也没有出来结果。

其实改进的方法并不复杂。上述方法之所以慢是因为重复的计算太多，只要避免重复计算就行了。比如我们可以把已经得到的数列中间项保存起来，如果下次需要计算的时候我们先查找一下，如果前面已经计算过了就不用再次计算了。

更简单的办法是从下往上计算，首先根据 f(0)和 f(1)算出 f(2)，在根据 f(1)和 f(2)算出 f(3)……依此类推就可以算出第 n 项了。很容易理解，这种思路的时间复杂度是 O(n)。

```
//////////////////////////////////////////////////////////////////////
//
// Calculate the nth item of Fibonacci Series iteratively
//////////////////////////////////////////////////////////////////////
//
long long Fibonacci_Solution2(unsigned n)
{
    int result[2] = {0, 1};
    if(n < 2)
        return result[n];

    long long  fibNMinusOne = 1;
```

```
    long long  fibNMinusTwo = 0;
    long long  fibN = 0;
    for(unsigned int i = 2; i <= n; ++ i)
    {
        fibN = fibNMinusOne + fibNMinusTwo;

        fibNMinusTwo = fibNMinusOne;
        fibNMinusOne = fibN;
    }

    return fibN;
}
```

这还不是最快的方法。下面介绍一种时间复杂度是 O(logn)的方法。在介绍这种方法之前，先介绍一个数学公式：

$\{f(n), f(n-1), f(n-1), f(n-2)\} = \{1, 1, 1,0\}^{n-1}$

(注：{f(n+1), f(n), f(n), f(n-1)}表示一个矩阵。在矩阵中第一行第一列是 f(n+1)，第一行第二列是 f(n)，第二行第一列是 f(n)，第二行第二列是 f(n-1)。)

有了这个公式，要求得 f(n)，我们只需要求得矩阵{1, 1, 1,0}的 n-1 次方，因为矩阵{1, 1, 1,0}的 n-1 次方的结果的第一行第一列就是 f(n)。这个数学公式用数学归纳法不难证明。感兴趣的朋友不妨自己证明一下。

现在的问题转换为求矩阵{1, 1, 1, 0}的乘方。如果简单第从 0 开始循环，n 次方将需要 n 次运算，并不比前面的方法要快。但我们可以考虑乘方的如下性质：

$$a^n = \begin{cases} a^{n/2} * a^{n/2} & n \text{ 为偶数时} \\ a^{(n-1)/2} * a^{(n-1)/2} & n \text{ 为奇数时} \end{cases}$$

要求得 n 次方，我们先求得 n/2 次方，再把 n/2 的结果平方一下。如果把求 n 次方的问题看成一个大问题，把求 n/2 看成一个较小的问题。这种把大问题分解成一个或多个小问题的思路我们称之为分治法。这样求 n 次方就只需要 logn 次运算了。

实现这种方式时，首先需要定义一个 2×2 的矩阵，并且定义好矩阵的乘法以及乘方运算。当这些运算定义好了之后，剩下的事情就变得非常简单。完整的实现代码如下所示。

```
#include <cassert>

//////////////////////////////////////////////////////////////////////
//
// A 2 by 2 matrix
//////////////////////////////////////////////////////////////////////
//
```

```
struct Matrix2By2
{
    Matrix2By2
    (
        long long m00 = 0,
        long long m01 = 0,
        long long m10 = 0,
        long long m11 = 0
    )
    :m_00(m00), m_01(m01), m_10(m10), m_11(m11)
    {
    }

    long long m_00;
    long long m_01;
    long long m_10;
    long long m_11;
};

///////////////////////////////////////////////////////////////////////
//
// Multiply two matrices
// Input: matrix1 - the first matrix
//       matrix2 - the second matrix
//Output: the production of two matrices
///////////////////////////////////////////////////////////////////////
//
Matrix2By2 MatrixMultiply
(
    const Matrix2By2& matrix1,
    const Matrix2By2& matrix2
)
{
    return Matrix2By2(
        matrix1.m_00 * matrix2.m_00 + matrix1.m_01 * matrix2.m_10,
        matrix1.m_00 * matrix2.m_01 + matrix1.m_01 * matrix2.m_11,
        matrix1.m_10 * matrix2.m_00 + matrix1.m_11 * matrix2.m_10,
        matrix1.m_10 * matrix2.m_01 + matrix1.m_11 * matrix2.m_11);
}

///////////////////////////////////////////////////////////////////////
//
// The nth power of matrix
// 1  1
```

```
// 1  0
////////////////////////////////////////////////////////////////////////
//
Matrix2By2 MatrixPower(unsigned int n)
{
    assert(n > 0);

    Matrix2By2 matrix;
    if(n == 1)
    {
        matrix = Matrix2By2(1, 1, 1, 0);
    }
    else if(n % 2 == 0)
    {
        matrix = MatrixPower(n / 2);
        matrix = MatrixMultiply(matrix, matrix);
    }
    else if(n % 2 == 1)
    {
        matrix = MatrixPower((n - 1) / 2);
        matrix = MatrixMultiply(matrix, matrix);
        matrix = MatrixMultiply(matrix, Matrix2By2(1, 1, 1, 0));
    }

    return matrix;
}

////////////////////////////////////////////////////////////////////////
//
// Calculate the nth item of Fibonacci Series using devide and conquer
////////////////////////////////////////////////////////////////////////
//
long long Fibonacci_Solution3(unsigned int n)
{
    int result[2] = {0, 1};
    if(n < 2)
        return result[n];

    Matrix2By2 PowerNMinus2 = MatrixPower(n - 1);
    return PowerNMinus2.m_00;
}
```

程序员面试题精选 100 题(17)－把字符串转换成整数

题目：输入一个表示整数的字符串，把该字符串转换成整数并输出。例如输入字符串"345"，则输出整数 345。

分析：这道题尽管不是很难，学过 C/C++语言一般都能实现基本功能，但不同程序员就这道题写出的代码有很大区别，可以说这道题能够很好地反应出程序员的思维和编程习惯，因此已经被包括微软在内的多家公司用作面试题。建议读者在往下看之前自己先编写代码，再比较自己写的代码和下面的参考代码有哪些不同。

首先我们分析如何完成基本功能，即如何把表示整数的字符串正确地转换成整数。还是以"345"作为例子。当我们扫描到字符串的第一个字符'3'时，我们不知道后面还有多少位，仅仅知道这是第一位，因此此时得到的数字是 3。当扫描到第二个数字'4'时，此时我们已经知道前面已经一个 3 了，再在后面加上一个数字 4，那前面的 3 相当于 30，因此得到的数字是 3*10+4=34。接着我们又扫描到字符'5'，我们已经知道了'5'的前面已经有了 34，由于后面要加上一个 5，前面的 34 就相当于 340 了，因此得到的数字就是 34*10+5=345。

分析到这里，我们不能得出一个转换的思路：每扫描到一个字符，我们把在之前得到的数字乘以 10 再加上当前字符表示的数字。这个思路用循环不难实现。

由于整数可能不仅仅之含有数字，还有可能以'+'或者'-'开头，表示整数的正负。因此我们需要把这个字符串的第一个字符做特殊处理。如果第一个字符是'+'号，则不需要做任何操作；如果第一个字符是'-'号，则表明这个整数是个负数，在最后的时候我们要把得到的数值变成负数。

接着我们试着处理非法输入。由于输入的是指针，在使用指针之前，我们要做的第一件是判断这个指针是不是为空。如果试着去访问空指针，将不可避免地导致程序崩溃。另外，输入的字符串中可能含有不是数字的字符。每当碰到这些非法的字符，我们就没有必要再继续转换。最后一个需要考虑的问题是溢出问题。由于输入的数字是以字符串的形式输入，因此有可能输入一个很大的数字转换之后会超过能够表示的最大的整数而溢出。

现在已经分析的差不多了，开始考虑编写代码。首先我们考虑如何声明这个函数。由于是把字符串转换成整数，很自然我们想到：

```
int StrToInt(const char* str);
```

这样声明看起来没有问题。但当输入的字符串是一个空指针或者含有非法的字符时，应该返回什么值呢？0 怎么样？那怎么区分非法输入和字符串本身就是"0"这两种情况呢？

接下来我们考虑另外一种思路。我们可以返回一个布尔值来指示输入是否有效，而把转换后的整数放到参数列表中以引用或者指针的形式传入。于是我们就可以声明如下：

```
bool StrToInt(const char *str, int& num);
```

这种思路解决了前面的问题。但是这个函数的用户使用这个函数的时候会觉得不是很方便，因为他不能直接把得到的整数赋值给其他整形变脸，显得不够直观。

前面的第一种声明就很直观。如何在保证直观的前提下当碰到非法输入的时候通知用户呢？一种解决方案就是定义一个全局变量，每当碰到非法输入的时候，就标记该全局变量。用户在调用这个函数之后，就可以检验该全局变量来判断转换是不是成功。

下面我们写出完整的实现代码。参考代码：

```
enum Status {kValid = 0, kInvalid};
int g_nStatus = kValid;

//////////////////////////////////////////////////////////////////////
//
// Convert a string into an integer
//////////////////////////////////////////////////////////////////////
//
int StrToInt(const char* str)
{
    g_nStatus = kInvalid;
        longlongnum = 0;

    if(str != NULL)
    {
        const char* digit = str;

        // the first char in the string maybe '+' or '-'
        bool minus = false;
        if(*digit == '+')
            digit ++;
        else if(*digit == '-')
        {
            digit ++;
            minus = true;
        }

        // the remaining chars in the string
        while(*digit != '\0')
        {
            if(*digit >= '0' && *digit <= '9')
            {
                num = num * 10 + (*digit - '0');

                        // overflow

    if(num>std::numeric_limits<int>::max())
{
 num = 0;
```

```
break;
}

digit++;
}
            // if the char is not a digit, invalid input
            else
            {
                num = 0;
                break;
            }
        }

        if(*digit == '\0')
        {
            g_nStatus = kValid;
            if(minus)
                num = 0 - num;
        }
    }

      return static_cast<int>(num);
}
```

讨论：在参考代码中，我选用的是第一种声明方式。不过在面试时，我们可以选用任意一种声明方式进行实现。但当面试官问我们选择的理由时，我们要对两者的优缺点进行评价。第一种声明方式对用户而言非常直观，但使用了全局变量，不够优雅；而第二种思路是用返回值来表明输入是否合法，在很多 API 中都用这种方法，但该方法声明的函数使用起来不够直观。

最后值得一提的是，在 C 语言提供的库函数中，函数 atoi 能够把字符串转换整数。它的声明是 `int atoi(const char *str)`。该函数就是用一个全局变量来标志输入是否合法的。

程序员面试题精选 100 题(18)－用两个栈实现队列

题目：某队列的声明如下：

```
template<typename T> class CQueue
{
public:
    CQueue() {}
    ~CQueue() {}
```

```
    void appendTail(const T& node);  // append a element to tail
    void deleteHead();              // remove a element from head

private:
    T>m_stack1;
    T>m_stack2;
};
```

分析：从上面的类的声明中，我们发现在队列中有两个栈。因此这道题实质上是要求我们用两个栈来实现一个队列。相信大家对栈和队列的基本性质都非常了解了：栈是一种后入先出的数据容器，因此对队列进行的插入和删除操作都是在栈顶上进行；队列是一种先入先出的数据容器，我们总是把新元素插入到队列的尾部，而从队列的头部删除元素。

我们通过一个具体的例子来分析往该队列插入和删除元素的过程。首先插入一个元素 a，不妨把先它插入到 m_stack1。这个时候 m_stack1 中的元素有{a}，m_stack2 为空。再插入两个元素 b 和 c，还是插入到 m_stack1 中，此时 m_stack1 中的元素有{a,b,c}，m_stack2 中仍然是空的。

这个时候我们试着从队列中删除一个元素。按照队列先入先出的规则，由于 a 比 b、c 先插入到队列中，这次被删除的元素应该是 a。元素 a 存储在 m_stack1 中，但并不在栈顶上，因此不能直接进行删除。注意到 m_stack2 我们还一直没有使用过，现在是让 m_stack2 起作用的时候了。如果我们把 m_stack1 中的元素逐个 pop 出来并 push 进入 m_stack2，元素在 m_stack2 中的顺序正好和原来在 m_stack1 中的顺序相反。因此经过两次 pop 和 push 之后，m_stack1 为空，而 m_stack2 中的元素是{c,b,a}。这个时候就可以 pop 出 m_stack2 的栈顶 a 了。pop 之后的 m_stack1 为空，而 m_stack2 的元素为{c,b}，其中 b 在栈顶。

这个时候如果我们还想继续删除应该怎么办呢？在剩下的两个元素中 b 和 c，b 比 c 先进入队列，因此 b 应该先删除。而此时 b 恰好又在栈顶上，因此可以直接 pop 出去。这次 pop 之后，m_stack1 中仍然为空，而 m_stack2 为{c}。

从上面的分析我们可以总结出删除一个元素的步骤：当 m_stack2 中不为空时，在 m_stack2 中的栈顶元素是最先进入队列的元素，可以 pop 出去。如果 m_stack2 为空时，我们把 m_stack1 中的元素逐个 pop 出来并 push 进入 m_stack2。由于先进入队列的元素被压到 m_stack1 的底端，经过 pop 和 push 之后就处于 m_stack2 的顶端了，又可以直接 pop 出去。

接下来我们再插入一个元素 d。我们是不是还可以把它 push 进 m_stack1？这样会不会有问题呢？我们说不会有问题。因为在删除元素的时候，如果 m_stack2 中不为空，处于 m_stack2 中的栈顶元素是最先进入队列的，可以直接 pop；如果 m_stack2 为空，我们把 m_stack1 中的元素 pop 出来并 push 进入 m_stack2。由于 m_stack2 中元素的顺序和 m_stack1 相反，最先进入队列的元素还是处于 m_stack2 的栈顶，仍然可以直接 pop。不会出现任何矛盾。

我们用一个表来总结一下前面的例子执行的步骤：

操作	m_stack1	m_stack2

append a	{a}	{}
append b	{a,b}	{}
append c	{a,b,c}	{}
delete head	{}	{b,c}
delete head	{}	{c}
append d	{d}	{c}
delete head	{d}	{}

总结完 push 和 pop 对应的过程之后，我们可以开始动手写代码了。参考代码如下：

```
//////////////////////////////////////////////////////////////////////
//
// Append a element at the tail of the queue
//////////////////////////////////////////////////////////////////////
//
template<typename T> void CQueue<T>::appendTail(const T& element)
{
    // push the new element into m_stack1
    m_stack1.push(element);
}

//////////////////////////////////////////////////////////////////////
//
// Delete the head from the queue
//////////////////////////////////////////////////////////////////////
//
template<typename T> void CQueue<T>::deleteHead()
{
    // if m_stack2is empty,and there are some
//elements inm_stack1, push them in m_stack2
if(m_stack2.size()<= 0)
{
while(m_stack1.size()>0)
{
T&data =m_stack1.top();
m_stack1.pop();
m_stack2.push(data);
}
}

// push theelement into m_stack2
assert(m_stack2.size()>0);
m_stack2.pop();
}
```

扩展：这道题是用两个栈实现一个队列。反过来能不能用两个队列实现一个栈？如果可以，该如何实现？

程序员面试题精选 100 题(19)－反转链表

题目：输入一个链表的头结点，反转该链表，并返回反转后链表的头结点。链表结点定义如下：

```
struct ListNode
{
    int      m_nKey;
    ListNode* m_pNext;
};
```

分析：这是一道广为流传的微软面试题。由于这道题能够很好的反应出程序员思维是否严密，在微软之后已经有很多公司在面试时采用了这道题。

为了正确地反转一个链表，需要调整指针的指向。与指针操作相关代码总是容易出错的，因此最好在动手写程序之前作全面的分析。在面试的时候不急于动手而是一开始做仔细的分析和设计，将会给面试官留下很好的印象，因为在实际的软件开发中，设计的时间总是比写代码的时间长。与其很快地写出一段漏洞百出的代码，远不如用较多的时间写出一段健壮的代码。

为了将调整指针这个复杂的过程分析清楚，我们可以借助图形来直观地分析。假设下图中 l、m 和 n 是三个相邻的结点：

a←b←...←l m→n→...

假设经过若干操作，我们已经把结点 l 之前的指针调整完毕，这些结点的 m_pNext 指针都指向前面一个结点。现在我们遍历到结点 m。当然，我们需要把调整结点的 m_pNext 指针让它指向结点 l。但注意一旦调整了指针的指向，链表就断开了，如下图所示：

a←b←...l←m n→...

因为已经没有指针指向结点 n，我们没有办法再遍历到结点 n 了。因此为了避免链表断开，我们需要在调整 m 的 m_pNext 之前要把 n 保存下来。

接下来我们试着找到反转后链表的头结点。不难分析出反转后链表的头结点是原始链表的尾位结点。什么结点是尾结点？就是 m_pNext 为空指针的结点。

基于上述分析，我们不难写出如下代码：

```
////////////////////////////////////////////////////////////////////
//
```

```
// Reverse a list iteratively
// Input: pHead - the head of the original list
// Output: the head of the reversed head
////////////////////////////////////////////////////////////////////
//
ListNode* ReverseIteratively(ListNode* pHead)
{
    ListNode* pReversedHead = NULL;
    ListNode* pNode = pHead;
    ListNode* pPrev = NULL;
    while(pNode != NULL)
    {
        // get the next node, and save it at pNext
        ListNode* pNext = pNode->m_pNext;

        // if the next node is null, the currect is the end of original
        // list, and it's the head of the reversed list
        if(pNext == NULL)
            pReversedHead = pNode;

        // reverse the linkage between nodes
        pNode->m_pNext = pPrev;

        // move forward on the the list
        pPrev = pNode;
        pNode = pNext;
    }

    return pReversedHead;
}
```

扩展：本题也可以递归实现。感兴趣的读者请自己编写递归代码。

程序员面试题精选 100 题(20)－最长公共子串

题目：如果字符串一的所有字符按其在字符串中的顺序出现在另外一个字符串二中，则字符串一称之为字符串二的子串。注意，并不要求子串（字符串一）的字符必须连续出现在字符串二中。请编写一个函数，输入两个字符串，求它们的最长公共子串，并打印出最长公共子串。

例如：输入两个字符串 BDCABA 和 ABCBDAB，字符串 BCBA 和 BDAB 都是是它们的最长公共子串，则输出它们的长度 4，并打印任意一个子串。

分析：求最长公共子串（Longest Common Subsequence, LCS）是一道非常经典的动态规划题，因此一些重视算法的公司像 MicroStrategy 都把它当作面试题。

完整介绍动态规划将需要很长的篇幅，因此我不打算在此全面讨论动态规划相关的概念，只集中对 LCS 直接相关内容作讨论。如果对动态规划不是很熟悉，请参考相关算法书比如算法讨论。

先介绍 LCS 问题的性质：记 $X_m=\{x_0, x_1,\ldots x_{m-1}\}$和 $Y_n=\{y_0,y_1,\ldots,y_{n-1}\}$为两个字符串，而 $Z_k=\{z_0,z_1,\ldots z_{k-1}\}$是它们的 LCS，则：

1. 如果 $x_{m-1}=y_{n-1}$，那么 $z_{k-1}=x_{m-1}=y_{n-1}$，并且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的 LCS；
2. 如果 $x_{m-1}\neq y_{n-1}$，那么当 $z_{k-1}\neq x_{m-1}$ 时 Z 是 X_{m-1} 和 Y 的 LCS；
3. 如果 $x_{m-1}\neq y_{n-1}$，那么当 $z_{k-1}\neq y_{n-1}$ 时 Z 是 Y_{n-1} 和 X 的 LCS；

下面简单证明一下这些性质：

1. 如果 $z_{k-1}\neq x_{m-1}$，那么我们可以把 x_{m-1}（y_{n-1}）加到 Z 中得到 Z’，这样就得到 X 和 Y 的一个长度为 k+1 的公共子串 Z’。这就与长度为 k 的 Z 是 X 和 Y 的 LCS 相矛盾了。因此一定有 $z_{k-1}=x_{m-1}=y_{n-1}$。

既然 $z_{k-1}=x_{m-1}=y_{n-1}$，那如果我们删除 z_{k-1}（x_{m-1}、y_{n-1}）得到的 Z_{k-1}，X_{m-1} 和 Y_{n-1}，显然 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的一个公共子串，现在我们证明 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的 LCS。用反证法不难证明。假设有 X_{m-1} 和 Y_{n-1} 有一个长度超过 k-1 的公共子串 W，那么我们把加到 W 中得到 W’，那 W’就是 X 和 Y 的公共子串，并且长度超过 k，这就和已知条件相矛盾了。

2. 还是用反证法证明。假设 Z 不是 X_{m-1} 和 Y 的 LCS，则存在一个长度超过 k 的 W 是 X_{m-1} 和 Y 的 LCS，那 W 肯定也 X 和 Y 的公共子串，而已知条件中 X 和 Y 的公共子串的最大长度为 k。矛盾。

3. 证明同 2。

有了上面的性质，我们可以得出如下的思路：求两字符串 $X_m=\{x_0, x_1,\ldots x_{m-1}\}$和 $Y_n=\{y_0,y_1,\ldots,y_{n-1}\}$的 LCS，如果 $x_{m-1}=y_{n-1}$，那么只需求得 X_{m-1} 和 Y_{n-1} 的 LCS，并在其后添加 x_{m-1}（y_{n-1}）即可；如果 $x_{m-1}\neq y_{n-1}$，我们分别求得 X_{m-1} 和 Y 的 LCS 和 Y_{n-1} 和 X 的 LCS，并且这两个 LCS 中较长的一个为 X 和 Y 的 LCS。

如果我们记字符串 X_i 和 Y_j 的 LCS 的长度为 c[i,j]，我们可以递归地求 c[i,j]：

$$c[i,j]=\begin{cases}0 & \text{if } i<0 \text{ or } j<0\\ c[i-1,j-1]+1 & \text{if } i,j>=0 \text{ and } x_i=x_j\\ \max(c[i,j-1],c[i-1,j]) & \text{if } i,j>=0 \text{ and } x_i\neq x_j\end{cases}$$

上面的公式用递归函数不难求得。但从前面求 Fibonacci 第 n 项(本面试题系列第 16 题）的分析中我们知道直接递归会有很多重复计算，我们用从底向上循环求解的思路效率更高。

为了能够采用循环求解的思路，我们用一个矩阵（参考代码中的 LCS_length）保存下来当前已经计算好了的 c[i,j]，当后面的计算需要这些数据时就可以直接从矩阵读取。另外，求取 c[i,j]可以从 c[i-1,j-1]、c[i,j-1]

或者 c[i-1,j]三个方向计算得到，相当于在矩阵 LCS_length 中是从 c[i-1,j-1]，c[i,j-1]或者 c[i-1,j]的某一个各自移动到 c[i,j]，因此在矩阵中有三种不同的移动方向：向左、向上和向左上方，其中只有向左上方移动时才表明找到 LCS 中的一个字符。于是我们需要用另外一个矩阵（参考代码中的 LCS_direction）保存移动的方向。

参考代码如下：

```
#include "string.h"

// directions of LCS generation
enum decreaseDir {kInit = 0, kLeft, kUp, kLeftUp};

/////////////////////////////////////////////////////////////////////////
////////
// Get the length of two strings' LCSs, and print one of the LCSs
// Input: pStr1        - the first string
//       pStr2        - the second string
// Output: the length of two strings' LCSs
/////////////////////////////////////////////////////////////////////////
////////
int LCS(char* pStr1, char* pStr2)
{
    if(!pStr1 || !pStr2)
        return 0;

size_t length1 = strlen(pStr1);
    size_t length2 = strlen(pStr2);
    if(!length1 || !length2)
        return 0;

size_t i, j;

// initiate the length matrix
    int **LCS_length;
    LCS_length = (int**)(new int[length1]);
    for(i = 0; i < length1; ++ i)
        LCS_length[i] = (int*)new int[length2];

    for(i = 0; i < length1; ++ i)
        for(j = 0; j < length2; ++ j)
            LCS_length[i][j] = 0;
```

```
// initiate the direction matrix
int **LCS_direction;
LCS_direction = (int**)(new int[length1]);
for( i = 0; i < length1; ++ i)
    LCS_direction[i] = (int*)new int[length2];

for(i = 0; i < length1; ++ i)
    for(j = 0; j < length2; ++ j)
        LCS_direction[i][j] = kInit;

for(i = 0; i < length1; ++ i)
{
    for(j = 0; j < length2; ++ j)
    {
        if(i == 0 || j == 0)
        {
            if(pStr1[i] == pStr2[j])
            {
                LCS_length[i][j] = 1;
                LCS_direction[i][j] = kLeftUp;
            }
            else
                LCS_length[i][j] = 0;
        }
        // a char of LCS is found,
        // it comes from the left up entry in the direction matrix
        else if(pStr1[i] == pStr2[j])
        {
            LCS_length[i][j] = LCS_length[i - 1][j - 1] + 1;
            LCS_direction[i][j] = kLeftUp;
        }
        // it comes from the up entry in the direction matrix
        else if(LCS_length[i - 1][j] > LCS_length[i][j - 1])
        {
            LCS_length[i][j] = LCS_length[i - 1][j];
            LCS_direction[i][j] = kUp;
        }
        // it comes from the left entry in the direction matrix
        else
        {
            LCS_length[i][j] = LCS_length[i][j - 1];
            LCS_direction[i][j] = kLeft;
        }
    }
```

```
    }
    LCS_Print(LCS_direction, pStr1, pStr2, length1 - 1, length2 - 1);

    return LCS_length[length1 - 1][length2 - 1];
}



////////////////////////////////////////////////////////////////////////
////////
// Print a LCS for two strings
// Input: LCS_direction - a 2d matrix which records the direction of
//                        LCS generation
//       pStr1        - the first string
//       pStr2        - the second string
//       row          - the row index in the matrix LCS_direction
//       col          - the column index in the matrix LCS_direction
////////////////////////////////////////////////////////////////////////
////////
void LCS_Print(int **LCS_direction,
            char* pStr1, char* pStr2,
            size_t row, size_t col)
{
    if(pStr1 == NULL || pStr2 == NULL)
        return;

    size_t length1 = strlen(pStr1);
    size_t length2 = strlen(pStr2);

if(length1 == 0 || length2 == 0 || !(row < length1 && col < length2))
        return;

    // kLeftUp implies a char in the LCS is found
    if(LCS_direction[row][col] == kLeftUp)
    {
        if(row > 0 && col > 0)
            LCS_Print(LCS_direction, pStr1, pStr2, row - 1, col - 1);

// print the char
        printf("%c", pStr1[row]);
    }
    else if(LCS_direction[row][col] == kLeft)
    {
        // move to the left entry in the direction matrix
        if(col > 0)
```

```
            LCS_Print(LCS_direction, pStr1, pStr2, row, col - 1);
    }
    else if(LCS_direction[row][col] == kUp)
    {
        // move to the up entry in the direction matrix
        if(row > 0)
            LCS_Print(LCS_direction, pStr1, pStr2, row - 1, col);
    }
}
```

扩展：如果题目改成求两个字符串的最长公共子字符串，应该怎么求？子字符串的定义和子串的定义类似，但要求是连续分布在其他字符串中。比如输入两个字符串 BDCABA 和 ABCBDAB 的最长公共字符串有 BD 和 AB，它们的长度都是 2。

程序员面试题精选 100 题(21)－左旋转字符串

题目：定义字符串的左旋转操作：把字符串前面的若干个字符移动到字符串的尾部。如把字符串 abcdef 左旋转 2 位得到字符串 cdefab。请实现字符串左旋转的函数。要求时间对长度为 n 的字符串操作的复杂度为 O(n)，辅助内存为 O(1)。

分析：如果不考虑时间和空间复杂度的限制，最简单的方法莫过于把这道题看成是把字符串分成前后两部分，通过旋转操作把这两个部分交换位置。于是我们可以新开辟一块长度为 n+1 的辅助空间，把原字符串后半部分拷贝到新空间的前半部分，在把原字符串的前半部分拷贝到新空间的后半部分。不难看出，这种思路的时间复杂度是 O(n)，需要的辅助空间也是 O(n)。

接下来的一种思路可能要稍微麻烦一点。我们假设把字符串左旋转 m 位。于是我们先把第 0 个字符保存起来，把第 m 个字符放到第 0 个的位置，在把第 2m 个字符放到第 m 个的位置…依次类推，一直移动到最后一个可以移动字符，最后在把原来的第 0 个字符放到刚才移动的位置上。接着把第 1 个字符保存起来，把第 m+1 个元素移动到第 1 个位置…重复前面处理第 0 个字符的步骤，直到处理完前面的 m 个字符。

该思路还是比较容易理解，但当字符串的长度 n 不是 m 的整数倍的时候，写程序会有些麻烦，感兴趣的朋友可以自己试一下。由于下面还要介绍更好的方法，这种思路的代码我就不提供了。

我们还是把字符串看成有两段组成的，记位 XY。左旋转相当于要把字符串 XY 变成 YX。我们先在字符串上定义一种翻转的操作，就是翻转字符串中字符的先后顺序。把 X 翻转后记为 X^T。显然有$(X^T)^T=X$。

我们首先对 X 和 Y 两段分别进行翻转操作，这样就能得到 X^TY^T。接着再对 X^TY^T 进行翻转操作，得到$(X^TY^T)^T=(Y^T)^T(X^T)^T=YX$。正好是我们期待的结果。

分析到这里我们再回到原来的题目。我们要做的仅仅是把字符串分成两段，第一段为前面 m 个字符，其余的字符分到第二段。再定义一个翻转字符串的函数，按照前面的步骤翻转三次就行了。时间复杂度和空间复杂度都合乎要求。

参考代码如下：

```
#include "string.h"

//////////////////////////////////////////////////////////////////////
//
// Move the first n chars in a string to its end
//////////////////////////////////////////////////////////////////////
//
char* LeftRotateString(char* pStr, unsigned int n)
{
    if(pStr != NULL)
    {
        int nLength = static_cast<int>(strlen(pStr));
        if(nLength > 0 || n == 0 || n > nLength)
        {
            char* pFirstStart = pStr;
            char* pFirstEnd = pStr + n - 1;
            char* pSecondStart = pStr + n;
            char* pSecondEnd = pStr + nLength - 1;

            // reverse the first part of the string
            ReverseString(pFirstStart, pFirstEnd);
            // reverse the second part of the strint
            ReverseString(pSecondStart, pSecondEnd);
            // reverse the whole string
            ReverseString(pFirstStart, pSecondEnd);
        }
    }

    return pStr;
}

//////////////////////////////////////////////////////////////////////
//
// Reverse the string between pStart and pEnd
//////////////////////////////////////////////////////////////////////
//
void ReverseString(char* pStart, char* pEnd)
{
    if(pStart == NULL || pEnd == NULL)
```

```
    {
        while(pStart <= pEnd)
        {
            char temp = *pStart;
            *pStart = *pEnd;
            *pEnd = temp;

            pStart ++;
            pEnd --;
        }
    }
}
```

PS：$(X^TY^T)^T=(Y^T)^T(X^T)^T=YX$

程序员面试题精选 100 题(22)－整数的二进制表示中 1 的个数

题目：输入一个整数，求该整数的二进制表达中有多少个 1。例如输入 10，由于其二进制表示为 1010，有两个 1，因此输出 2。

分析：这是一道很基本的考查位运算的面试题。包括微软在内的很多公司都曾采用过这道题。

一个很基本的想法是，我们先判断整数的最右边一位是不是 1。接着把整数右移一位，原来处于右边第二位的数字现在被移到第一位了，再判断是不是 1。这样每次移动一位，直到这个整数变成 0 为止。现在的问题变成怎样判断一个整数的最右边一位是不是 1 了。很简单，如果它和整数 1 作与运算。由于 1 除了最右边一位以外，其他所有位都为 0。因此如果与运算的结果为 1，表示整数的最右边一位是 1，否则是 0。

得到的代码如下：

```
//////////////////////////////////////////////////////////////////////
//
// Get how many 1s in an integer's binary expression
//////////////////////////////////////////////////////////////////////
//
int NumberOf1_Solution1(int i)
{
    int count = 0;
    while(i)
    {
```

```
        if(i & 1)
            count ++;

        i = i >> 1;
    }

    return count;
}
```

可能有读者会问，整数右移一位在数学上是和除以 2 是等价的。那可不可以把上面的代码中的右移运算符换成除以 2 呢？答案是最好不要换成除法。因为除法的效率比移位运算要低的多，在实际编程中如果可以应尽可能地用移位运算符代替乘除法。

这个思路当输入 i 是正数时没有问题，但当输入的 i 是一个负数时，不但不能得到正确的 1 的个数，还将导致死循环。以负数 0x80000000 为例，右移一位的时候，并不是简单地把最高位的 1 移到第二位变成 0x40000000，而是 0xC0000000。这是因为移位前是个负数，仍然要保证移位后是个负数，因此移位后的最高位会设为 1。如果一直做右移运算，最终这个数字就会变成 0xFFFFFFFF 而陷入死循环。

为了避免死循环，我们可以不右移输入的数字 i。首先 i 和 1 做与运算，判断 i 的最低位是不是为 1。接着把 1 左移一位得到 2，再和 i 做与运算，就能判断 i 的次高位是不是 1……这样反复左移，每次都能判断 i 的其中一位是不是 1。基于此，我们得到如下代码：

```
////////////////////////////////////////////////////////////////////////
//
// Get how many 1s in an integer's binary expression
////////////////////////////////////////////////////////////////////////
//
int NumberOf1_Solution2(int i)
{
    int count = 0;
    unsigned int flag = 1;
    while(flag)
    {
        if(i & flag)
            count ++;

        flag = flag << 1;
    }

    return count;
}
```

另外一种思路是如果一个整数不为 0，那么这个整数至少有一位是 1。如果我们把这个整数减去 1，那么原来处在整数最右边的 1 就会变成 0，原来在 1 后面的所有的 0 都会变成 1。其余的所

有位将不受到影响。举个例子：一个二进制数 1100，从右边数起的第三位是处于最右边的一个 1。减去 1 后，第三位变成 0，它后面的两位 0 变成 1，而前面的 1 保持不变，因此得到结果是 1011。

我们发现减 1 的结果是把从最右边一个 1 开始的所有位都取反了。这个时候如果我们再把原来的整数和减去 1 之后的结果做与运算，从原来整数最右边一个 1 那一位开始所有位都会变成 0。如 1100&1011=1000。也就是说，把一个整数减去 1，再和原整数做与运算，会把该整数最右边一个 1 变成 0。那么一个整数的二进制有多少个 1，就可以进行多少次这样的操作。

这种思路对应的代码如下：

```
////////////////////////////////////////////////////////////////////////
//
// Get how many 1s in an integer's binary expression
////////////////////////////////////////////////////////////////////////
//
int NumberOf1_Solution3(int i)
{
    int count = 0;

    while (i)
    {
        ++ count;
        i = (i - 1) & i;
    }

    return count;
}
```

扩展：如何用一个语句判断一个整数是不是二的整数次幂？

PS：n&(n-1)==0;//二进制数只有一位位 1，则该数是 2 的整数次幂.

程序员面试题精选 100 题(23)－跳台阶问题

题目：一个台阶总共有 n 级，如果一次可以跳 1 级，也可以跳 2 级。求总共有多少总跳法，并分析算法的时间复杂度。

分析：这道题最近经常出现，包括 MicroStrategy 等比较重视算法的公司都曾先后选用过个这道题作为面试题或者笔试题。

首先我们考虑最简单的情况。如果只有 1 级台阶，那显然只有一种跳法。如果有 2 级台阶，那就有两种跳的方法了：一种是分两次跳，每次跳 1 级；另外一种就是一次跳 2 级。

现在我们再来讨论一般情况。我们把 n 级台阶时的跳法看成是 n 的函数，记为 f(n)。当 n>2 时，第一次跳的时候就有两种不同的选择：一是第一次只跳 1 级，此时跳法数目等于后面剩下的 n-1 级台阶的跳法数目，即为 f(n-1)；另外一种选择是第一次跳 2 级，此时跳法数目等于后面剩下的 n-2 级台阶的跳法数目，即为 f(n-2)。因此 n 级台阶时的不同跳法的总数 f(n)=f(n-1)+(f-2)。

我们把上面的分析用一个公式总结如下：

```
        /  1                        n=1
f(n)=       2                        n=2
        \  f(n-1)+(f-2)               n>2
```

分析到这里，相信很多人都能看出这就是我们熟悉的 Fibonacci 序列。至于怎么求这个序列的第 n 项，请参考本面试题系列第 16 题，这里就不在赘述了。

程序员面试题精选 100 题(24)－栈的 push、pop 序列

题目：输入两个整数序列。其中一个序列表示栈的 push 顺序，判断另一个序列有没有可能是对应的 pop 顺序。为了简单起见，我们假设 push 序列的任意两个整数都是不相等的。

比如输入的 push 序列是 1、2、3、4、5，那么 4、5、3、2、1 就有可能是一个 pop 系列。因为可以有如下的 push 和 pop 序列：push 1，push 2，push 3，push 4，pop，push 5，pop，pop，pop，pop，这样得到的 pop 序列就是 4、5、3、2、1。但序列 4、3、5、1、2 就不可能是 push 序列 1、2、3、4、5 的 pop 序列。

分析：这到题除了考查对栈这一基本数据结构的理解，还能考查我们的分析能力。

这道题的一个很直观的想法就是建立一个辅助栈，每次 push 的时候就把一个整数 push 进入这个辅助栈，同样需要 pop 的时候就把该栈的栈顶整数 pop 出来。

我们以前面的序列 4、5、3、2、1 为例。第一个希望被 pop 出来的数字是 4，因此 4 需要先 push 到栈里面。由于 push 的顺序已经由 push 序列确定了，也就是在把 4 push 进栈之前，数字 1，2，3 都需要 push 到栈里面。此时栈里的包含 4 个数字，分别是 1，2，3，4，其中 4 位于栈顶。把 4 pop 出栈后，剩下三个数字 1，2，3。接下来希望被 pop 的是 5，由于仍然不是栈顶数字，我们接着在 push 序列中 4 以后的数字中寻找。找到数字 5 后再一次 push 进栈，这个时候 5 就是位于栈顶，可以被 pop 出来。接下来希望被 pop 的三个数字是 3，2，1。每次操作前都位于栈顶，直接 pop 即可。

再来看序列 4、3、5、1、2。pop 数字 4 的情况和前面一样。把 4 pop 出来之后，3 位于栈顶，直接 pop。接下来希望 pop 的数字是 5，由于 5 不是栈顶数字，我们到 push 序列中没有被 push 进栈的数字中去搜索该数字，幸运的时候能够找到 5，于是把 5 push 进入栈。此时 pop 5 之后，栈内包含两个数字 1、2，其中

2 位于栈顶。这个时候希望 pop 的数字是 1，由于不是栈顶数字，我们需要到 push 序列中还没有被 push 进栈的数字中去搜索该数字。但此时 push 序列中所有数字都已被 push 进入栈，因此该序列不可能是一个 pop 序列。

也就是说，如果我们希望 pop 的数字正好是栈顶数字，直接 pop 出栈即可；如果希望 pop 的数字目前不在栈顶，我们就到 push 序列中还没有被 push 到栈里的数字中去搜索这个数字，并把在它之前的所有数字都 push 进栈。如果所有的数字都被 push 进栈仍然没有找到这个数字，表明该序列不可能是一个 pop 序列。

基于前面的分析，我们可以写出如下的参考代码：

```
#include <stack>

//////////////////////////////////////////////////////////////////////
////////
// Given a push order of a stack, determine whether an array is possible
to
// be its corresponding pop order
// Input: pPush   - an array of integers, the push order
//        pPop    - an array of integers, the pop order
//        nLength - the length of pPush and pPop
// Output: If pPop is possible to be the pop order of pPush, return true.
//         Otherwise return false
//////////////////////////////////////////////////////////////////////
////////
bool IsPossiblePopOrder(const int* pPush, const int* pPop, int nLength)
{
    bool bPossible = false;

    if(pPush && pPop && nLength > 0)
    {
        const int *pNextPush = pPush;
        const int *pNextPop = pPop;

        // ancillary stack
        std::stack<int>stackData;

        // check every integers in pPop
        while(pNextPop - pPop < nLength)
        {
            // while the top of the ancillary stack is not the integer
            // to be poped, try to push some integers into the stack
            while(stackData.empty() || stackData.top() != *pNextPop)
            {
                // pNextPush == NULL means all integers have been
                // pushed into the stack, can't push any longer
```

```
            if(!pNextPush)
                break;

            stackData.push(*pNextPush);

            // if there are integers left in pPush, move
            // pNextPush forward, otherwise set it to be NULL
            if(pNextPush - pPush < nLength - 1)
                pNextPush ++;
            else
                pNextPush = NULL;
        }

        // After pushing, the top of stack is still not same as
        // pPextPop, pPextPop is not in a pop sequence
        // corresponding to pPush
        if(stackData.top() != *pNextPop)
            break;

        // Check the next integer in pPop
        stackData.pop();
        pNextPop ++;
    }

    // if all integers in pPop have been check successfully,
    // pPop is a pop sequence corresponding to pPush
    if(stackData.empty() && pNextPop - pPop == nLength)
        bPossible = true;
}

return bPossible;
}
```

程序员面试题精选 100 题(25)-在从 1 到 n 的正数中 1 出现的次数

题目：输入一个整数 n，求从 1 到 n 这 n 个整数的十进制表示中 1 出现的次数。

例如输入 12，从 1 到 12 这些整数中包含 1 的数字有 1，10，11 和 12，1 一共出现了 5 次。

分析：这是一道广为流传的 google 面试题。用最直观的方法求解并不是很难，但遗憾的是效率不是很高；而要得出一个效率较高的算法，需要比较强的分析能力，并不是件很容易的事情。当然，google 的面试题中简单的也没有几道。

首先我们来看最直观的方法，分别求得 1 到 n 中每个整数中 1 出现的次数。而求一个整数的十进制表示中 1 出现的次数，就和本面试题系列的第 22 题很相像了。我们每次判断整数的个位数字是不是 1。如果这个数字大于 10，除以 10 之后再判断个位数字是不是 1。基于这个思路，不难写出如下的代码：

```
int NumberOf1(unsigned int n);

////////////////////////////////////////////////////////////////////////
////////
// Find the number of 1 in the integers between 1 and n
// Input: n - an integer
// Output: the number of 1 in the integers between 1 and n
////////////////////////////////////////////////////////////////////////
////////
int NumberOf1BeforeBetween1AndN_Solution1(unsigned int n)
{
    int number = 0;

    // Find the number of 1 in each integer between 1 and n
    for(unsigned int i = 1; i <= n; ++ i)
        number += NumberOf1(i);

    return number;
}

////////////////////////////////////////////////////////////////////////
////////
// Find the number of 1 in an integer with radix 10
// Input: n - an integer
// Output: the number of 1 in n with radix
////////////////////////////////////////////////////////////////////////
////////
int NumberOf1(unsigned int n)
{
    int number = 0;
    while(n)
    {
        if(n % 10 == 1)
            number ++;

        n = n / 10;
    }
```

```
    return number;
}
```

这个思路有一个非常明显的缺点就是每个数字都要计算 1 在该数字中出现的次数，因此时间复杂度是 O(n)。当输入的 n 非常大的时候，需要大量的计算，运算效率很低。我们试着找出一些规律，来避免不必要的计算。

我们用一个稍微大一点的数字 21345 作为例子来分析。我们把从 1 到 21345 的所有数字分成两段，即 1-1235 和 1346-21345。

先来看 1346-21345 中 1 出现的次数。1 的出现分为两种情况：一种情况是 1 出现在最高位（万位）。从 1 到 21345 的数字中，1 出现在 10000-19999 这 10000 个数字的万位中，一共出现了 10000（10^4）次；另外一种情况是 1 出现在除了最高位之外的其他位中。例子中 1346-21345，这 20000 个数字中后面四位中 1 出现的次数是 2000 次（$2*10^3$，其中 2 的第一位的数值，10^3 是因为数字的后四位数字其中一位为 1，其余的三位数字可以在 0 到 9 这 10 个数字任意选择，由排列组合可以得出总次数是 $2*10^3$）。

至于从 1 到 1345 的所有数字中 1 出现的次数，我们就可以用递归地求得了。这也是我们为什么要把 1-21345 分为 1-1235 和 1346-21345 两段的原因。因为把 21345 的最高位去掉就得到 1345，便于我们采用递归的思路。

分析到这里还有一种特殊情况需要注意：前面我们举例子是最高位是一个比 1 大的数字，此时最高位 1 出现的次数 10^4（对五位数而言）。但如果最高位是 1 呢？比如输入 12345，从 10000 到 12345 这些数字中，1 在万位出现的次数就不是 10^4 次，而是 2346 次了，也就是除去最高位数字之后剩下的数字再加上 1。

基于前面的分析，我们可以写出以下的代码。在参考代码中，为了编程方便，我把数字转换成字符串了。

```
#include "string.h"
#include "stdlib.h"

int NumberOf1(const char* strN);
int PowerBase10(unsigned int n);

////////////////////////////////////////////////////////////////////////
////////
// Find the number of 1 in an integer with radix 10
// Input: n - an integer
// Output: the number of 1 in n with radix
////////////////////////////////////////////////////////////////////////
////////
int NumberOf1BeforeBetween1AndN_Solution2(int n)
{
    if(n <= 0)
        return 0;
```

```
    // convert the integer into a string
    char strN[50];
    sprintf(strN, "%d", n);

    return NumberOf1(strN);
}

////////////////////////////////////////////////////////////////////////
////////
// Find the number of 1 in an integer with radix 10
// Input: strN - a string, which represents an integer
// Output: the number of 1 in n with radix
////////////////////////////////////////////////////////////////////////
////////
int NumberOf1(const char* strN)
{
    if(!strN || *strN < '0' || *strN > '9' || *strN == '\0')
        return 0;

    int firstDigit = *strN - '0';
    unsigned int length = static_cast<unsigned int>(strlen(strN));

    // the integer contains only one digit
    if(length == 1 && firstDigit == 0)
        return 0;

    if(length == 1 && firstDigit > 0)
        return 1;

    // suppose the integer is 21345
    // numFirstDigit is the number of 1 of 10000-19999 due to the first
digit
    int numFirstDigit = 0;
    // numOtherDigits is the number of 1 01346-21345 due to all digits
    // except the first one
    int numOtherDigits = firstDigit * (length - 1) * PowerBase10(length
- 2);
    // numRecursive is the number of 1 of integer 1345
    int numRecursive = NumberOf1(strN + 1);

    // if the first digit is greater than 1, suppose in integer 21345
    // number of 1 due to the first digit is 10^4. It's 10000-19999
    if(firstDigit > 1)
        numFirstDigit = PowerBase10(length - 1);
```

```
    // if the first digit equals to 1, suppose in integer 12345
    // number of 1 due to the first digit is 2346. It's 10000-12345
    else if(firstDigit == 1)
        numFirstDigit = atoi(strN + 1) + 1;

    return numFirstDigit + numOtherDigits + numRecursive;
}

////////////////////////////////////////////////////////////////////////
////////
// Calculate 10^n
////////////////////////////////////////////////////////////////////////
////////
int PowerBase10(unsigned int n)
{
    int result = 1;
    for(unsigned int i = 0; i < n; ++ i)
        result *= 10;

    return result;
}
```

程序员面试题精选 100 题(26)-和为 n 连续正数序列

题目：输入一个正数 n，输出所有和为 n 连续正数序列。

例如输入 15，由于 1+2+3+4+5=4+5+6=7+8=15，所以输出 3 个连续序列 1-5、4-6 和 7-8。

分析：这是网易的一道面试题。

这道题和本面试题系列的第 10 题有些类似。我们用两个数 small 和 big 分别表示序列的最小值和最大值。首先把 small 初始化为 1，big 初始化为 2。如果从 small 到 big 的序列的和大于 n 的话，我们向右移动 small，相当于从序列中去掉较小的数字。如果从 small 到 big 的序列的和小于 n 的话，我们向右移动 big，相当于向序列中添加 big 的下一个数字。一直到 small 等于(1+n)/2，因为序列至少要有两个数字。

基于这个思路，我们可以写出如下代码：

```
void PrintContinuousSequence(int small, int big);

////////////////////////////////////////////////////////////////////////
```

```
////
// Find continuous sequence, whose sum is n
////////////////////////////////////////////////////////////////////////
////
void FindContinuousSequence(int n)
{
    if(n < 3)
        return;

    int small = 1;
    int big = 2;
    int middle = (1 + n) / 2;
    int sum = small + big;

    while(small < middle)
    {
        // we are lucky and find the sequence
        if(sum == n)
            PrintContinuousSequence(small, big);

        // if the current sum is greater than n,
        // move small forward
        while(sum > n)
        {
            sum -= small;
            small ++;

            // we are lucky and find the sequence
            if(sum == n)
                PrintContinuousSequence(small, big);
        }

        // move big forward
        big ++;
        sum += big;
    }
}

////////////////////////////////////////////////////////////////////////
////
// Print continuous sequence between small and big
////////////////////////////////////////////////////////////////////////
////
void PrintContinuousSequence(int small, int big)
```

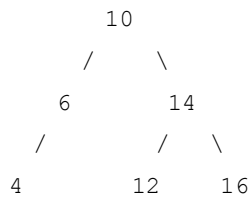
```
{
    for(int i = small; i <= big; ++ i)
        printf("%d ", i);

    printf("\n");
}
```

程序员面试题精选 100 题(27)-二元树的深度

题目：输入一棵二元树的根结点，求该树的深度。从根结点到叶结点依次经过的结点（含根、叶结点）形成树的一条路径，最长路径的长度为树的深度。

例如：输入二元树：



输出该树的深度 3。

二元树的结点定义如下：

```
struct SBinaryTreeNode // a node of the binary tree
{
    int             m_nValue; // value of node
    SBinaryTreeNode  *m_pLeft;  // left child of node
    SBinaryTreeNode  *m_pRight; // right child of node
};
```

分析：这道题本质上还是考查二元树的遍历。

题目给出了一种树的深度的定义。当然，我们可以按照这种定义去得到树的所有路径，也就能得到最长路径以及它的长度。只是这种思路用来写程序有点麻烦。

我们还可以从另外一个角度来理解树的深度。如果一棵树只有一个结点，它的深度为 1。如果根结点只有左子树而没有右子树，那么树的深度应该是其左子树的深度加 1；同样如果根结点只有右子树而没有左子树，那么树的深度应该是其右子树的深度加 1。如果既有右子树又有左子树呢？那该树的深度就是其左、右子树深度的较大值再加 1。

上面的这个思路用递归的方法很容易实现，只需要对遍历的代码稍作修改即可。参考代码如下：

```
//////////////////////////////////////////////////////////////////////
//
// Get depth of a binary tree
// Input: pTreeNode - the head of a binary tree
// Output: the depth of a binary tree
//////////////////////////////////////////////////////////////////////
//
int TreeDepth(SBinaryTreeNode *pTreeNode)
{
    // the depth of a empty tree is 0
    if(!pTreeNode)
        return 0;

    // the depth of left sub-tree
    int nLeft = TreeDepth(pTreeNode->m_pLeft);
    // the depth of right sub-tree
    int nRight = TreeDepth(pTreeNode->m_pRight);

    // depth is the binary tree
    return (nLeft > nRight) ? (nLeft + 1) : (nRight + 1);
}
```

程序员面试题精选 100 题(28)-字符串的排列

题目：输入一个字符串，打印出该字符串中字符的所有排列。例如输入字符串 abc，则输出由字符 a、b、c 所能排列出来的所有字符串 abc、acb、bac、bca、cab 和 cba。

分析：这是一道很好的考查对递归理解的编程题，因此在过去一年中频繁出现在各大公司的面试、笔试题中。

我们以三个字符 abc 为例来分析一下求字符串排列的过程。首先我们固定第一个字符 a，求后面两个字符 bc 的排列。当两个字符 bc 的排列求好之后，我们把第一个字符 a 和后面的 b 交换，得到 bac，接着我们固定第一个字符 b，求后面两个字符 ac 的排列。现在是把 c 放到第一位置的时候了。记住前面我们已经把原先的第一个字符 a 和后面的 b 做了交换，为了保证这次 c 仍然是和原先处在第一位置的 a 交换，我们在拿 c 和第一个字符交换之前，先要把 b 和 a 交换回来。在交换 b 和 a 之后，再拿 c 和处在第一位置的 a 进行交换，得到 cba。我们再次固定第一个字符 c，求后面两个字符 b、a 的排列。

既然我们已经知道怎么求三个字符的排列，那么固定第一个字符之后求后面两个字符的排列，就是典型的递归思路了。

基于前面的分析，我们可以得到如下的参考代码：

```
void Permutation(char* pStr, char* pBegin);

/////////////////////////////////////////////////////////////////////
////
// Get the permutation of a string,
// for example, input string abc, its permutation is
// abc acb bac bca cba cab
/////////////////////////////////////////////////////////////////////
////
void Permutation(char* pStr)
{
    Permutation(pStr, pStr);
}

/////////////////////////////////////////////////////////////////////
////
// Print the permutation of a string,
// Input: pStr   - input string
//       pBegin - points to the begin char of string
//              which we want to permutate in this recursion
/////////////////////////////////////////////////////////////////////
////
void Permutation(char* pStr, char* pBegin)
{
    if(!pStr || !pBegin)
        return;

    // if pBegin points to the end of string,
    // this round of permutation is finished,
    // print the permuted string
    if(*pBegin == '\0')
    {
        printf("%s\n", pStr);
    }
    // otherwise, permute string
    else
    {
        for(char* pCh = pBegin; *pCh != '\0'; ++ pCh)
        {
            // swap pCh and pBegin
            char temp = *pCh;
            *pCh = *pBegin;
```

```
            *pBegin = temp;

            Permutation(pStr, pBegin + 1);

            // restore pCh and pBegin  PS:改变字符串顺序后必须还原回来!
            temp = *pCh;
            *pCh = *pBegin;
            *pBegin = temp;
        }
    }
}
```

扩展 1：如果不是求字符的所有排列，而是求字符的所有组合，应该怎么办呢？当输入的字符串中含有相同的字符串时，相同的字符交换位置是不同的排列，但是同一个组合。举个例子，如果输入 aaa，那么它的排列是 6 个 aaa，但对应的组合只有一个。

扩展 2：输入一个含有 8 个数字的数组，判断有没有可能把这 8 个数字分别放到正方体的 8 个顶点上，使得正方体上三组相对的面上的 4 个顶点的和相等。

程序员面试题精选 100 题(29)-调整数组顺序使奇数位于偶数前面

题目：输入一个整数数组，调整数组中数字的顺序，使得所有奇数位于数组的前半部分，所有偶数位于数组的后半部分。要求时间复杂度为 O(n)。

分析：如果不考虑时间复杂度，最简单的思路应该是从头扫描这个数组，每碰到一个偶数时，拿出这个数字，并把位于这个数字后面的所有数字往前挪动一位。挪完之后在数组的末尾有一个空位，这时把该偶数放入这个空位。由于碰到一个偶数，需要移动 O(n)个数字，因此总的时间复杂度是 $O(n^2)$。

要求的是把奇数放在数组的前半部分，偶数放在数组的后半部分，因此所有的奇数应该位于偶数的前面。也就是说我们在扫描这个数组的时候，如果发现有偶数出现在奇数的前面，我们可以交换他们的顺序，交换之后就符合要求了。

因此我们可以维护两个指针，第一个指针初始化为数组的第一个数字，它只向后移动；第二个指针初始化为数组的最后一个数字，它只向前移动。在两个指针相遇之前，第一个指针总是位于第二个指针的前面。如果第一个指针指向的数字是偶数而第二个指针指向的数字是奇数，我们就交换这两个数字。

基于这个思路，我们可以写出如下的代码：

```
void Reorder(int *pData, unsigned int length, bool (*func)(int));
bool isEven(int n);

////////////////////////////////////////////////////////////////////////
////
// Devide an array of integers into two parts, odd in the first part,
// and even in the second part
// Input: pData  - an array of integers
//        length - the length of array
////////////////////////////////////////////////////////////////////////
////
void ReorderOddEven(int *pData, unsigned int length)
{
    if(pData == NULL || length == 0)
        return;

    Reorder(pData, length, isEven);
}

////////////////////////////////////////////////////////////////////////
////
// Devide an array of integers into two parts, the intergers which
// satisfy func in the first part, otherwise in the second part
// Input: pData  - an array of integers
//        length - the length of array
//        func   - a function
////////////////////////////////////////////////////////////////////////
////
void Reorder(int *pData, unsigned int length, bool (*func)(int))
{
    if(pData == NULL || length == 0)
        return;

    int *pBegin = pData;
    int *pEnd = pData + length - 1;

    while(pBegin < pEnd)
    {
        // if *pBegin does not satisfy func, move forward
        if(!func(*pBegin))
        {
            pBegin ++;
            continue;
        }
```

```
        // if *pEnd does not satisfy func, move backward
        if(func(*pEnd))
        {
            pEnd --;
            continue;
        }

        // if *pBegin satisfy func while *pEnd does not,
        // swap these integers
        int temp = *pBegin;
        *pBegin = *pEnd;
        *pEnd = temp;
    }
}

//////////////////////////////////////////////////////////////////////
////
// Determine whether an integer is even or not
// Input: an integer
// otherwise return false
//////////////////////////////////////////////////////////////////////
////
bool isEven(int n)
{
    return (n & 1) == 0;
}
```

讨论：

上面的代码有三点值得提出来和大家讨论：

1．函数 isEven 判断一个数字是不是偶数并没有用%运算符而是用&。理由是通常情况下位运算符比%要快一些；

2．这道题有很多变种。这里要求是把奇数放在偶数的前面，如果把要求改成：把负数放在非负数的前面等，思路都是都一样的。

3．在函数 Reorder 中，用函数指针 func 指向的函数来判断一个数字是不是符合给定的条件，而不是用在代码直接判断（hard code）。这样的好处是把调整顺序的算法和调整的标准分开了（即解耦，decouple）。当调整的标准改变时，Reorder 的代码不需要修改，只需要提供一个新的确定调整标准的函数即可，提高了代码的可维护性。例如要求把负数放在非负数的前面，我们不需要修改 Reorder 的代码，只需添加一个函数来判断整数是不是非负数。这样的思路在很多库中都有广泛的应用，比如在ＳＴＬ的很多算法函数中都有一个仿函数（functor）的参数（当然仿函数不是

函数指针，但其思想是一样的）。如果在面试中能够想到这一层，无疑能给面试官留下很好的印象。

程序员面试题精选 100 题(30)-异常安全的赋值运算符重载函数

题目：类 CMyString 的声明如下：

```
class CMyString
{
public:
    CMyString(char* pData = NULL);
    CMyString(const CMyString& str);
    ~CMyString(void);
    CMyString& operator = (const CMyString& str);

private:
    char* m_pData;
};
```

请实现其赋值运算符的重载函数，要求异常安全，即当对一个对象进行赋值时发生异常，对象的状态不能改变。

分析：首先我们来看一般 C++教科书上给出的赋值运算符的重载函数：

```
CMyString& CMyString::operator =(const CMyString &str)
{
    if(this == &str)
        return *this;

    delete []m_pData;
    m_pData = NULL;

    m_pData = new char[strlen(str.m_pData) + 1];
    strcpy(m_pData, str.m_pData);

    return *this;
}
```

我们知道，在分配内存时有可能发生异常。当执行语句 new char[strlen(str.m_pData) + 1] 发生异常时，程序将从该赋值运算符的重载函数退出不再执行。注意到这个时候语句 delete []m_pData 已经执行了。也就是说赋值操作没有完成，但原来对象的状态已经改变。也就是说不满足题目的异常安全的要求。

为了满足异常安全这个要求，一个简单的办法是掉换 new、delete 的顺序。先把内存 new 出来用一个临时指针保存起来，只有这个语句正常执行完成之后再执行 delete。这样就能够保证异常安全了。

下面给出的是一个更加优雅的实现方案：

```
CMyString& CMyString::operator =(const CMyString &str)
{
    if(this != &str)
    {
        CMyString strTemp(str);

        char* pTemp = strTemp.m_pData;
        strTemp.m_pData = m_pData;
        m_pData = pTemp;
    }

    return *this;
}
```

该方案通过调用构造拷贝函数创建一个临时对象来分配内存。此时即使发生异常，对原来对象的状态没有影响。交换临时对象和需要赋值的对象的字符串指针之后，由于临时对象的生命周期结束，自动调用其析构函数释放需赋值对象的原来的字符串空间。整个函数不需要显式用到 new、delete，内存的分配和释放都自动完成，因此代码显得比较优雅。

程序员面试题精选 100 题(31)-从尾到头输出链表

题目：输入一个链表的头结点，从尾到头反过来输出每个结点的值。链表结点定义如下：

```
struct ListNode
{
    int      m_nKey;
    ListNode* m_pNext;
};
```

分析：这是一道很有意思的面试题。该题以及它的变体经常出现在各大公司的面试、笔试题中。

看到这道题后，第一反应是从头到尾输出比较简单。于是很自然地想到把链表中链接结点的指针反转过来，改变链表的方向。然后就可以从头到尾输出了。反转链表的算法详见本人面试题精选系列的第 19 题，在此不再细述。但该方法需要额外的操作，应该还有更好的方法。

接下来的想法是从头到尾遍历链表，每经过一个结点的时候，把该结点放到一个栈中。当遍历完整个链表后，再从栈顶开始输出结点的值，此时输出的结点的顺序已经反转过来了。该方法需要维护一个额外的栈，实现起来比较麻烦。

既然想到了栈来实现这个函数，而递归本质上就是一个栈结构。于是很自然的又想到了用递归来实现。要实现反过来输出链表，我们每访问到一个结点的时候，先递归输出它后面的结点，再输出该结点自身，这样链表的输出结果就反过来了。

基于这样的思路，不难写出如下代码：

```
//////////////////////////////////////////////////////////////////////
//
// Print a list from end to beginning
// Input: pListHead - the head of list
//////////////////////////////////////////////////////////////////////
//
void PrintListReversely(ListNode* pListHead)
{
    if(pListHead != NULL)
    {
        // Print the next node first
        if (pListHead->m_pNext != NULL)
        {
            PrintListReversely(pListHead->m_pNext);
        }

        // Print this node
        printf("%d", pListHead->m_nKey);
    }
}
```

扩展：该题还有两个常见的变体：

1. 从尾到头输出一个字符串；
2. 定义一个函数求字符串的长度，要求该函数体内不能声明任何变量。

程序员面试题精选 100 题(32)-不能被继承的类

题目：用 C++设计一个不能被继承的类。

分析：这是 Adobe 公司 2007 年校园招聘的最新笔试题。这道题除了考察应聘者的 C++基本功底外，还能考察反应能力，是一道很好的题目。

在 Java 中定义了关键字 final，被 final 修饰的类不能被继承。但在 C++中没有 final 这个关键字，要实现这个要求还是需要花费一些精力。

首先想到的是在 C++ 中，子类的构造函数会自动调用父类的构造函数。同样，子类的析构函数也会自动调用父类的析构函数。要想一个类不能被继承，我们只要把它的构造函数和析构函数都定义为私有函数。那么当一个类试图从它那继承的时候，必然会由于试图调用构造函数、析构函数而导致编译错误。

可是这个类的构造函数和析构函数都是私有函数了，我们怎样才能得到该类的实例呢？这难不倒我们，我们可以通过定义静态来创建和释放类的实例。基于这个思路，我们可以写出如下的代码：

```
//////////////////////////////////////////////////////////////////////
//
// Define a class which can't be derived from
//////////////////////////////////////////////////////////////////////
//
class FinalClass1
{
public:
    static FinalClass1* GetInstance()
    {
        return new FinalClass1;
    }

    static void DeleteInstance( FinalClass1* pInstance)
    {
        delete pInstance;
        pInstance = 0;
    }

private:
    FinalClass1() {}
    ~FinalClass1() {}
};
```

这个类是不能被继承，但在总觉得它和一般的类有些不一样，使用起来也有点不方便。比如，我们只能得到位于堆上的实例，而得不到位于栈上实例。

能不能实现一个和一般类除了不能被继承之外其他用法都一样的类呢？办法总是有的，不过需要一些技巧。请看如下代码：

```
//////////////////////////////////////////////////////////////////////
//
// Define a class which can't be derived from
//////////////////////////////////////////////////////////////////////
//
template <typename T> class MakeFinal
{
    friend T;

private:
    MakeFinal() {}
    ~MakeFinal() {}
};

class FinalClass2 : virtual public MakeFinal<FinalClass2>
{
public:
    FinalClass2() {}
    ~FinalClass2() {}
};
```

这个类使用起来和一般的类没有区别，可以在栈上、也可以在堆上创建实例。尽管类 MakeFinal<FinalClass2>的构造函数和析构函数都是私有的，但由于类 FinalClass2 是它的友元函数，因此在 FinalClass2 中调用 MakeFinal<FinalClass2>的构造函数和析构函数都不会造成编译错误。

但当我们试图从 FinalClass2 继承一个类并创建它的实例时，却不同通过编译。

```
class Try : public FinalClass2
{
public:
    Try() {}
    ~Try() {}
};

Try temp;
```

由于类 FinalClass2 是从类 MakeFinal<FinalClass2>虚继承过来的，在调用 Try 的构造函数的时候，会直接跳过 FinalClass2 而直接调用 MakeFinal<FinalClass2> 的构造函数。非常遗憾的是，Try 不是 MakeFinal<FinalClass2>的友元，因此不能调用其私有的构造函数。

基于上面的分析，试图从 FinalClass2 继承的类，一旦实例化，都会导致编译错误，因此是 FinalClass2 不能被继承。这就满足了我们设计要求。

程序员面试题精选 100 题(33)-在 O(1)时间删除链表结点

题目：给定链表的头指针和一个结点指针，在 O(1)时间删除该结点。链表结点的定义如下：

```
struct ListNode
{
    int       m_nKey;
    ListNode*  m_pNext;
};
```

函数的声明如下：

```
void DeleteNode(ListNode* pListHead, ListNode* pToBeDeleted);
```

分析：这是一道广为流传的 Google 面试题，能有效考察我们的编程基本功，还能考察我们的反应速度，更重要的是，还能考察我们对时间复杂度的理解。

在链表中删除一个结点，最常规的做法是从链表的头结点开始，顺序查找要删除的结点，找到之后再删除。由于需要顺序查找，时间复杂度自然就是 O(n) 了。

我们之所以需要从头结点开始查找要删除的结点，是因为我们需要得到要删除的结点的前面一个结点。我们试着换一种思路。我们可以从给定的结点得到它的下一个结点。这个时候我们实际删除的是它的下一个结点，由于我们已经得到实际删除的结点的前面一个结点，因此完全是可以实现的。当然，在删除之前，我们需要需要把给定的结点的下一个结点的数据拷贝到给定的结点中。此时，时间复杂度为 O(1)。

上面的思路还有一个问题：如果删除的结点位于链表的尾部，没有下一个结点，怎么办？我们仍然从链表的头结点开始，顺便遍历得到给定结点的前序结点，并完成删除操作。这个时候时间复杂度是 O(n)。

那题目要求我们需要在 O(1)时间完成删除操作，我们的算法是不是不符合要求？实际上，假设链表总共有 n 个结点，我们的算法在 n-1 总情况下时间复杂度是 O(1)，只有当给定的结点处于链表末尾的时候，时间复杂度为 O(n)。那么平均时间复杂度[(n-1)*O(1)+O(n)]/n，仍然为 O(1)。

基于前面的分析，我们不难写出下面的代码。

参考代码：

```
////////////////////////////////////////////////////////////////////////
//
// Delete a node in a list
// Input: pListHead - the head of list
//       pToBeDeleted - the node to be deleted
////////////////////////////////////////////////////////////////////////
//
void DeleteNode(ListNode* pListHead, ListNode* pToBeDeleted)
{
    if(!pListHead || !pToBeDeleted)
        return;

    // if pToBeDeleted is not the last node in the list
    if(pToBeDeleted->m_pNext != NULL)
    {
        // copy data from the node next to pToBeDeleted
        ListNode* pNext = pToBeDeleted->m_pNext;
        pToBeDeleted->m_nKey = pNext->m_nKey;
        pToBeDeleted->m_pNext = pNext->m_pNext;

        // delete the node next to the pToBeDeleted
        delete pNext;
        pNext = NULL;
    }
    // if pToBeDeleted is the last node in the list
    else
    {
        // get the node prior to pToBeDeleted
        ListNode* pNode = pListHead;
        while(pNode->m_pNext != pToBeDeleted)
        {
            pNode = pNode->m_pNext;
        }

        // deleted pToBeDeleted
        pNode->m_pNext = NULL;
        delete pToBeDeleted;
        pToBeDeleted = NULL;
    }
}
```

值得注意的是，为了让代码看起来简洁一些，上面的代码基于两个假设：（1）给定的结点的确在链表中；（2）给定的要删除的结点不是链表的头结点。不考虑第一个假设对代码的鲁棒性是有影响的。至于第二个假设，当整个列表只有一个结点时，代码会有问题。但这个假设不算很过分，因为在有些链表的实现中，

会创建一个虚拟的链表头，并不是一个实际的链表结点。这样要删除的结点就不可能是链表的头结点了。当然，在面试中，我们可以把这些假设和面试官交流。这样，面试官还是会觉得我们考虑问题很周到的。

PS:假设要删除 A 节点，A.next==B。则删除 B，并将 B 的值赋给 A，这样就等于删除了 A 节点。

程序员面试题精选 100 题(34)-找出数组中两个只出现一次的数字

题目：一个整型数组里除了两个数字之外，其他的数字都出现了两次。请写程序找出这两个只出现一次的数字。要求时间复杂度是 O(n)，空间复杂度是 O(1)。

分析：这是一道很新颖的关于位运算的面试题。

首先我们考虑这个问题的一个简单版本：一个数组里除了一个数字之外，其他的数字都出现了两次。请写程序找出这个只出现一次的数字。

这个题目的突破口在哪里？题目为什么要强调有一个数字出现一次，其他的出现两次？我们想到了异或运算的性质：任何一个数字异或它自己都等于 0。也就是说，如果我们从头到尾依次异或数组中的每一个数字，那么最终的结果刚好是那个只出现依次的数字，因为那些出现两次的数字全部在异或中抵消掉了。

有了上面简单问题的解决方案之后，我们回到原始的问题。如果能够把原数组分为两个子数组。在每个子数组中，包含一个只出现一次的数字，而其他数字都出现两次。如果能够这样拆分原数组，按照前面的办法就是分别求出这两个只出现一次的数字了。

我们还是从头到尾依次异或数组中的每一个数字，那么最终得到的结果就是两个只出现一次的数字的异或结果。因为其他数字都出现了两次，在异或中全部抵消掉了。由于这两个数字肯定不一样，那么这个异或结果肯定不为 0，也就是说在这个结果数字的二进制表示中至少就有一位为 1。我们在结果数字中找到第一个为 1 的位的位置，记为第 N 位。现在我们以第 N 位是不是 1 为标准把原数组中的数字分成两个子数组，第一个子数组中每个数字的第 N 位都为 1，而第二个子数组的每个数字的第 N 位都为 0。

现在我们已经把原数组分成了两个子数组，每个子数组都包含一个只出现一次的数字，而其他数字都出现了两次。因此到此为止，所有的问题我们都已经解决。

基于上述思路，我们不难写出如下代码：

```
//////////////////////////////////////////////////////////////////////
//
```

```
// Find two numbers which only appear once in an array
// Input: data - an array contains two number appearing exactly once,
//             while others appearing exactly twice
//       length - the length of data
// Output: num1 - the first number appearing once in data
//        num2 - the second number appearing once in data
////////////////////////////////////////////////////////////////////////
//
void FindNumsAppearOnce(int data[], int length, int &num1, int &num2)
{
    if (length < 2)
        return;

    // get num1 ^ num2
    int resultExclusiveOR = 0;
    for (int i = 0; i < length; ++ i)
        resultExclusiveOR ^= data[i];

    // get index of the first bit, which is 1 in resultExclusiveOR
    unsigned int indexOf1 = FindFirstBitIs1(resultExclusiveOR);

    num1 = num2 = 0;
    for (int j = 0; j < length; ++ j)
    {
        // divide the numbers in data into two groups,
        // the indexOf1 bit of numbers in the first group is 1,
        // while in the second group is 0
        if(IsBit1(data[j], indexOf1))
            num1 ^= data[j];
        else
            num2 ^= data[j];
    }
}

////////////////////////////////////////////////////////////////////////
//
// Find the index of first bit which is 1 in num (assuming not 0)
////////////////////////////////////////////////////////////////////////
//
unsigned int FindFirstBitIs1(int num)
{
    int indexBit = 0;
    while (((num & 1) == 0) && (indexBit < 32))
    {
```

```
        num = num >> 1;
        ++ indexBit;
    }

    return indexBit;
}

//////////////////////////////////////////////////////////////////////
//
// Is the indexBit bit of num 1?
//////////////////////////////////////////////////////////////////////
//
bool IsBit1(int num, unsigned int indexBit)
{
    num = num >> indexBit;

    return (num & 1);
}
```

PS:关键词->相同数字异或结果为 0，并根据所有元素异或结果第一个出现 0 的位置把数组分为 2 部分！！！

程序员面试题精选 100 题(35)-找出两个链表的第一个公共结点

题目：两个单向链表，找出它们的第一个公共结点。

链表的结点定义为：

```
struct ListNode
{
    int      m_nKey;
    ListNode*   m_pNext;
};
```

分析：这是一道微软的面试题。微软非常喜欢与链表相关的题目，因此在微软的面试题中，链表出现的概率相当高。

如果两个单向链表有公共的结点，也就是说两个链表从某一结点开始，它们的 m_pNext 都指向同一个结点。但由于是单向链表的结点，每个结点只有一个 m_pNext，因此从第一个公共结点开始，之后它们所有结点都是重合的，不可能

再出现分叉。所以，两个有公共结点而部分重合的链表，拓扑形状看起来像一个 Y，而不可能像 X。

看到这个题目，第一反应就是蛮力法：在第一链表上顺序遍历每个结点。每遍历一个结点的时候，在第二个链表上顺序遍历每个结点。如果此时两个链表上的结点是一样的，说明此时两个链表重合，于是找到了它们的公共结点。如果第一个链表的长度为 m，第二个链表的长度为 n，显然，该方法的时间复杂度为 O(mn)。

接下来我们试着去寻找一个线性时间复杂度的算法。我们先把问题简化：如何判断两个单向链表有没有公共结点？前面已经提到，如果两个链表有一个公共结点，那么该公共结点之后的所有结点都是重合的。那么，它们的最后一个结点必然是重合的。因此，我们判断两个链表是不是有重合的部分，只要分别遍历两个链表到最后一个结点。如果两个尾结点是一样的，说明它们用重合；否则两个链表没有公共的结点。

在上面的思路中，顺序遍历两个链表到尾结点的时候，我们不能保证在两个链表上同时到达尾结点。这是因为两个链表不一定长度一样。但如果假设一个链表比另一个长 l 个结点，我们先在长的链表上遍历 l 个结点，之后再同步遍历，这个时候我们就能保证同时到达最后一个结点了。由于两个链表从第一个公共结点考试到链表的尾结点，这一部分是重合的。因此，它们肯定也是同时到达第一公共结点的。于是在遍历中，第一个相同的结点就是第一个公共的结点。

在这个思路中，我们先要分别遍历两个链表得到它们的长度，并求出两个长度之差。在长的链表上先遍历若干次之后，再同步遍历两个链表，知道找到相同的结点，或者一直到链表结束。此时，如果第一个链表的长度为 m，第二个链表的长度为 n，该方法的时间复杂度为 O(m+n)。

基于这个思路，我们不难写出如下的代码：

```
//////////////////////////////////////////////////////////////////////
//
// Find the first common node in the list with head pHead1 and
// the list with head pHead2
// Input: pHead1 - the head of the first list
//        pHead2 - the head of the second list
// Return: the first common node in two list. If there is no common
//         nodes, return NULL
//////////////////////////////////////////////////////////////////////
//
ListNode* FindFirstCommonNode( ListNode *pHead1, ListNode *pHead2)
{
    // Get the length of two lists
    unsigned int nLength1 = ListLength(pHead1);
    unsigned int nLength2 = ListLength(pHead2);
    int nLengthDif = nLength1 - nLength2;

    // Get the longer list
```

```
    ListNode *pListHeadLong = pHead1;
    ListNode *pListHeadShort = pHead2;
    if(nLength2 > nLength1)
    {
        pListHeadLong = pHead2;
        pListHeadShort = pHead1;
        nLengthDif = nLength2 - nLength1;
    }

    // Move on the longer list
    for(int i = 0; i < nLengthDif; ++ i)
        pListHeadLong = pListHeadLong->m_pNext;

    // Move on both lists
    while((pListHeadLong != NULL) &&
        (pListHeadShort != NULL) &&
        (pListHeadLong != pListHeadShort))
    {
        pListHeadLong = pListHeadLong->m_pNext;
        pListHeadShort = pListHeadShort->m_pNext;
    }

    // Get the first common node in two lists
    ListNode *pFisrtCommonNode = NULL;
    if(pListHeadLong == pListHeadShort)
        pFisrtCommonNode = pListHeadLong;

    return pFisrtCommonNode;
}

////////////////////////////////////////////////////////////////////
//
// Get the length of list with head pHead
// Input: pHead - the head of list
// Return: the length of list
////////////////////////////////////////////////////////////////////
//
unsigned int ListLength(ListNode* pHead)
{
    unsigned int nLength = 0;
    ListNode* pNode = pHead;
    while(pNode != NULL)
    {
        ++ nLength;
```

```
        pNode = pNode->m_pNext;
    }

    return nLength;
}
```

PS:两个有公共结点而部分重合的链表，拓扑形状看起来像一个 Y，而不可能像 X。即从公共节点开始之后的所有都相同！求出链表长度差，遍历差数目长的链表，之后同步遍历两链表，第一个相同的节点即是结果。

程序员面试题精选 100 题(36)-在字符串中删除特定的字符

题目：输入两个字符串，从第一字符串中删除第二个字符串中所有的字符。例如，输入”They are students.”和”aeiou”，则删除之后的第一个字符串变成”Thy r stdnts.”。

分析：这是一道微软面试题。在微软的常见面试题中，与字符串相关的题目占了很大的一部分，因为写程序操作字符串能很好的反映我们的编程基本功。

要编程完成这道题要求的功能可能并不难。毕竟，这道题的基本思路就是在第一个字符串中拿到一个字符，在第二个字符串中查找一下，看它是不是在第二个字符串中。如果在的话，就从第一个字符串中删除。但如何能够把效率优化到让人满意的程度，却也不是一件容易的事情。也就是说，如何在第一个字符串中删除一个字符，以及如何在第二字符串中查找一个字符，都是需要一些小技巧的。

首先我们考虑如何在字符串中删除一个字符。由于字符串的内存分配方式是连续分配的。我们从字符串当中删除一个字符，需要把后面所有的字符往前移动一个字节的位置。但如果每次删除都需要移动字符串后面的字符的话，对于一个长度为 n 的字符串而言，删除一个字符的时间复杂度为 O(n)。而对于本题而言，有可能要删除的字符的个数是 n，因此该方法就删除而言的时间复杂度为 $O(n^2)$。

事实上，我们并不需要在每次删除一个字符的时候都去移动后面所有的字符。我们可以设想，当一个字符需要被删除的时候，我们把它所占的位置让它后面的字符来填补，也就相当于这个字符被删除了。在具体实现中，我们可以定义两个指针(pFast 和 pSlow)，初始的时候都指向第一字符的起始位置。当 pFast 指向的字符是需要删除的字符，则 pFast 直接跳过，指向下一个字符。如果 pFast 指向的字符是不需要删除的字符，那么把 pFast 指向的字符赋值给 pSlow 指向的字符，并且 pFast 和 pStart 同时向后移动指向下一个字符。这样，前面被 pFast 跳过的字符相当于被删除了。用这种方法，整个删除在 O(n)时间内就可以完成。

接下来我们考虑如何在一个字符串中查找一个字符。当然，最简单的办法就是从头到尾扫描整个字符串。显然，这种方法需要一个循环，对于一个长度为 n 的字符串，时间复杂度是 O(n)。

由于字符的总数是有限的。对于八位的 char 型字符而言，总共只有 2^8=256 个字符。我们可以新建一个大小为 256 的数组，把所有元素都初始化为 0。然后对于字符串中每一个字符，把它的 ASCII 码映射成索引，把数组中该索引对应的元素设为 1 。这个时候，要查找一个字符就变得很快了：根据这个字符的 ASCII 码，在数组中对应的下标找到该元素，如果为 0，表示字符串中没有该字符，否则字符串中包含该字符。此时，查找一个字符的时间复杂度是 O(1)。其实，这个数组就是一个 hash 表。这种思路的详细说明，详见本面试题系列的第 13 题。

基于上述分析，我们可以写出如下代码：

```
//////////////////////////////////////////////////////////////////////
//
// Delete all characters in pStrDelete from pStrSource
//////////////////////////////////////////////////////////////////////
//
void DeleteChars(char* pStrSource, const char* pStrDelete)
{
    if(NULL == pStrSource || NULL == pStrDelete)
        return;

    // Initialize an array, the index in this array is ASCII value.
    // All entries in the array, whose index is ASCII value of a
    // character in the pStrDelete, will be set as 1.
    // Otherwise, they will be set as 0.
    const unsigned int nTableSize = 256;
    int hashTable[nTableSize];
    memset(hashTable, 0, sizeof(hashTable));

    const char* pTemp = pStrDelete;
    while ('\0' != *pTemp)
    {
        hashTable[*pTemp] = 1;
        ++ pTemp;
    }

    char* pSlow = pStrSource;
    char* pFast = pStrSource;
    while ('\0' != *pFast)
    {
        // if the character is in pStrDelete, move both pStart and
        // pEnd forward, and copy pEnd to pStart.
```

```
        // Otherwise, move only pEnd forward, and the character
        // pointed by pEnd is deleted
        if(1 != hashTable[*pFast])
        {
            *pSlow = *pFast;
            ++ pSlow;
        }

        ++pFast;
    }

    *pSlow = '\0';
}
```

PS：删除字符：使用两个临时指针变量。
查找字符串：hash

程序员面试题精选 100 题(37)-寻找丑数

题目：我们把只包含因子
2、3 和 5 的数称作丑数（Ugly Number）。例如 6、8 都是丑数，但 14 不是，因为它包含因子 7。习惯上我们把 1 当做是第一个丑数。求按从小到大的顺序的第 1500 个丑数。
分析：这是一道在网络上广为流传的面试题，据说 google 曾经采用过这道题。

所谓一个数 m 是另一个数 n 的因子，是指 n 能被 m 整除，也就是 n % m == 0。根据丑数的定义，丑数只能被 2、3 和 5 整除。也就是说如果一个数如果它能被 2 整除，我们把它连续除以 2；如果能被 3 整除，就连续除以 3；如果能被 5 整除，就除以连续 5。如果最后我们得到的是 1，那么这个数就是丑数，否则不是。

基于前面的分析，我们可以写出如下的函数来判断一个数是不是丑数：

```
bool IsUgly(int number)
{
    while(number % 2 == 0)
        number /= 2;
    while(number % 3 == 0)
        number /= 3;
    while(number % 5 == 0)
        number /= 5;

    return (number == 1) ? true : false;
}
```

接下来，我们只需要按顺序判断每一个整数是不是丑数，即：

```
int GetUglyNumber_Solution1(int index)
```

```
{
    if(index <= 0)
        return 0;

    int number = 0;
    int uglyFound = 0;
    while(uglyFound < index)
    {
        ++number;

        if(IsUgly(number))
        {
            ++uglyFound;
        }
    }

    return number;
}
```

我们只需要在函数 GetUglyNumber_Solution1 中传入参数 1500，就能得到第 1500 个丑数。该算法非常直观，代码也非常简洁，但最大的问题我们每个整数都需要计算。即使一个数字不是丑数，我们还是需要对它做求余数和除法操作。因此该算法的时间效率不是很高。

接下来我们换一种思路来分析这个问题，试图只计算丑数，而不在非丑数的整数上花费时间。根据丑数的定义，丑数应该是另一个丑数乘以 2、3 或者 5 的结果（1 除外）。因此我们可以创建一个数组，里面的数字是排好序的丑数。里面的每一个丑数是前面的丑数乘以 2、3 或者 5 得到的。

这种思路的关键在于怎样确保数组里面的丑数是排好序的。我们假设数组中已经有若干个丑数，排好序后存在数组中。我们把现有的最大丑数记做 M。现在我们来生成下一个丑数，该丑数肯定是前面某一个丑数乘以 2、3 或者 5 的结果。我们首先考虑把已有的每个丑数乘以 2。在乘以 2 的时候，能得到若干个结果小于或等于 M 的。由于我们是按照顺序生成的，小于或者等于 M 肯定已经在数组中了，我们不需再次考虑；我们还会得到若干个大于 M 的结果，但我们只需要第一个大于 M 的结果，因为我们希望丑数是按从小到大顺序生成的，其他更大的结果我们以后再说。我们把得到的第一个乘以 2 后大于 M 的结果，记为 M_2。同样我们把已有的每一个丑数乘以 3 和 5，能得到第一个大于 M 的结果 M_3 和 M_5。那么下一个丑数应该是 M_2、M_3 和 M_5 三个数的最小者。

前面我们分析的时候，提到把已有的每个丑数分别都乘以 2、3 和 5，事实上是不需要的，因为已有的丑数是按顺序存在数组中的。对乘以 2 而言，肯定存在某一个丑数 T_2，排在它之前的每一个丑数乘以 2 得到的结果都会小于已有最大的丑数，在它之后的每一个丑数乘以 2 得到的结果都会太大。我们只需要记下这个丑数的位置，同时每次生成新的丑数的时候，去更新这个 T_2。对乘以 3 和 5 而言，存在着同样的 T_3 和 T_5。

有了这些分析，我们不难写出如下的代码：

```
int GetUglyNumber_Solution2(int index)
{
    if(index <= 0)
        return 0;

    int *pUglyNumbers = new int[index];
```

```
    pUglyNumbers[0] = 1;
    int nextUglyIndex = 1;

    int *pMultiply2 = pUglyNumbers;
    int *pMultiply3 = pUglyNumbers;
    int *pMultiply5 = pUglyNumbers;

    while(nextUglyIndex < index)
    {
        int min = Min(*pMultiply2 * 2, *pMultiply3 * 3, *pMultiply5 * 5);
        pUglyNumbers[nextUglyIndex] = min;

        while(*pMultiply2 * 2 <= pUglyNumbers[nextUglyIndex])
            ++pMultiply2;
        while(*pMultiply3 * 3 <= pUglyNumbers[nextUglyIndex])
            ++pMultiply3;
        while(*pMultiply5 * 5 <= pUglyNumbers[nextUglyIndex])
            ++pMultiply5;

        ++nextUglyIndex;
    }

    int ugly = pUglyNumbers[nextUglyIndex - 1];
    delete[] pUglyNumbers;
    return ugly;
}

int Min(int number1, int number2, int number3)
{
    int min = (number1 < number2) ? number1 : number2;
    min = (min < number3) ? min : number3;

    return min;
}
```

和第一种思路相比，这种算法不需要在非丑数的整数上做任何计算，因此时间复杂度要低很多。感兴趣的读者可以分别统计两个函数 GetUglyNumber_Solution1(1500) 和 GetUglyNumber_Solution2(1500)的运行时间。当然我们也要指出，第二种算法由于要保存已经生成的丑数，因此需要一个数组，从而需要额外的内存。第一种算法是没有这样的内存开销的。

程序员面试题精选 100 题(38)-输出 1 到最大的 N 位数

题目：输入数字 n，按顺序输出从 1 最大的 n 位 10 进制数。比如输入 3，则输出 1、2、3 一直到最大的 3 位数即 999。

分析：这是一道很有意思的题目。看起来很简单，其实里面却有不少的玄机。

应聘者在解决这个问题的时候，最容易想到的方法是先求出最大的 n 位数是什么，然后用一个循环从 1 开始逐个输出。很快，我们就能写出如下代码：

```
// Print numbers from 1 to the maximum number with n digits, in order
void Print1ToMaxOfNDigits_1(int n)
{
    // calculate 10^n
    int number = 1;
    int i = 0;
    while(i++ < n)
        number *= 10;

    // print from 1 to (10^n - 1)
    for(i = 1; i < number; ++i)
        printf("%d\t", i);
}
```

初看之下，好像没有问题。但如果我们仔细分析这个问题，就能注意到这里没有规定 n 的范围，当我们求最大的 n 位数的时候，是不是有可能用整型甚至长整型都会溢出？

分析到这里，我们很自然的就想到我们需要表达一个大数。最常用的也是最容易实现的表达大数的方法是用字符串或者整型数组（当然不一定是最有效的）。

用字符串表达数字的时候，最直观的方法就是字符串里每个字符都是’0’到’9’之间的某一个字符，表示数字中的某一位。因为数字最大是 n 位的，因此我们需要一个 n+1 位字符串（最后一位为结束符号’\0’）。当实际数字不够 n 位的时候，在字符串的前半部分补零。这样，数字的个位永远都在字符串的末尾（除去结尾符号）。

首先我们把字符串中每一位数字都初始化为’0’。然后每一次对字符串表达的数字加 1，再输出。因此我们只需要做两件事：一是在字符串表达的数字上模拟加法。另外我们要把字符串表达的数字输出。值得注意的是，当数字不够 n 位的时候，我们在数字的前面补零。输出的时候这些补位的 0 不应该输出。比如输入 3 的时候，那么数字 98 以 098 的形式输出，就不符合我们的习惯了。

基于上述分析，我们可以写出如下代码：

```
// Print numbers from 1 to the maximum number with n digits, in order
void Print1ToMaxOfNDigits_2(int n)
{
    // 0 or minus numbers are invalid input
    if(n <= 0)
        return;

    // number is initialized as 0
    char *number = new char[n + 1];
    memset(number, '0', n);
    number[n] = '\0';

    while(!Increment(number))
```

```
    {
        PrintNumber(number);
    }

    delete []number;
}

// Increment a number. When overflow, return true; otherwise return false
bool Increment(char* number)
{
    bool isOverflow = false;
    int nTakeOver = 0;
    int nLength = strlen(number);

    // Increment (Add 1) operation begins from the end of number
    for(int i = nLength - 1; i >= 0; i --)
    {
        int nSum = number[i] - '0' + nTakeOver;
        if(i == nLength - 1)
            nSum ++;

        if(nSum >= 10)
        {
            if(i == 0)
                isOverflow = true;
            else
            {
                nSum -= 10;
                nTakeOver = 1;
                number[i] = '0' + nSum;
            }
        }
        else
        {
            number[i] = '0' + nSum;
            break;
        }
    }

    return isOverflow;
}

// Print number stored in string, ignore 0 at its beginning
// For example, print "0098" as "98"
```

```
void PrintNumber(char* number)
{
    bool isBeginning0 = true;
    int nLength = strlen(number);

    for(int i = 0; i < nLength; ++ i)
    {
        if(isBeginning0 && number[i] != '0')
            isBeginning0 = false;

        if(!isBeginning0)
        {
            printf("%c", number[i]);
        }
    }

    printf("\t");
}
```

第二种思路基本上和第一种思路相对应，只是把一个整型数值换成了字符串的表示形式。同时，值得提出的是，判断打印是否应该结束时，我没有调用函数 strcmp 比较字符串 number 和"99…999"（n 个 9）。这是因为 strcmp 的时间复杂度是 O(n)，而判断是否溢出的平均时间复杂度是 O(1)。

第二种思路虽然比较直观，但由于模拟了整数的加法，代码有点长。要在面试短短几十分钟时间里完整正确写出这么长代码，不是件容易的事情。接下来我们换一种思路来考虑这个问题。如果我们在数字前面补 0 的话，就会发现 n 位所有 10 进制数其实就是 n 个从 0 到 9 的全排列。也就是说，我们把数字的每一位都从 0 到 9 排列一遍，就得到了所有的 10 进制数。只是我们在输出的时候，数字排在前面的 0 我们不输出罢了。

全排列用递归很容易表达，数字的每一位都可能是 0 到 9 中的一个数，然后设置下一位。递归结束的条件是我们已经设置了数字的最后一位。

```
// Print numbers from 1 to the maximum number with n digits, in order
void Print1ToMaxOfNDigits_3(int n)
{
    // 0 or minus numbers are invalid input
    if(n <= 0)
        return;

    char* number = new char[n + 1];
    number[n] = '\0';

    for(int i = 0; i < 10; ++i)
    {
        // first digit can be 0 to 9
        number[0] = i + '0';

        Print1ToMaxOfNDigitsRecursively(number, n, 0);
```

```
        }

        delete[] number;
    }

    // length: length of number
    // index: current index of digit in number for this round
    void Print1ToMaxOfNDigitsRecursively(char* number, int length, int index)
    {
        // we have reached the end of number, print it
        if(index == length - 1)
        {
            PrintNumber(number);
            return;
        }

        for(int i = 0; i < 10; ++i)
        {
            // next digit can be 0 to 9
            number[index + 1] = i + '0';

            // go to the next digit
            Print1ToMaxOfNDigitsRecursively(number, length, index + 1);
        }
    }
```

函数 PrintNumber 和前面第二种思路中的一样，这里就不重复了。对比这两种思路，我们可以发现，递归能够用很简洁的代码来解决问题。

程序员面试题精选 100 题(39)-颠倒栈

题目：用递归颠倒一个栈。例如输入栈{1, 2, 3, 4, 5}，1 在栈顶。颠倒之后的栈为{5, 4, 3, 2, 1}，5 处在栈顶。

分析：乍一看到这道题目，第一反应是把栈里的所有元素逐一 pop 出来，放到一个数组里，然后在数组里颠倒所有元素，最后把数组中的所有元素逐一 push 进入栈。这时栈也就颠倒过来了。颠倒一个数组是一件很容易的事情。不过这种思路需要显示分配一个长度为 O(n)的数组，而且也没有充分利用递归的特性。

我们再来考虑怎么递归。我们把栈{1, 2, 3, 4, 5}看成由两部分组成：栈顶元素 1 和剩下的部分{2, 3, 4, 5}。如果我们能把{2, 3, 4, 5}颠倒过来，变成{5, 4, 3, 2}，然后在把原来的栈顶元素 1 放到底部，那么就整个栈就颠倒过来了，变成{5, 4, 3, 2, 1}。

接下来我们需要考虑两件事情：一是如何把{2, 3, 4, 5}颠倒过来变成{5, 4, 3, 2}。我们只要把{2, 3, 4, 5}看成由两部分组成：栈顶元素 2 和剩下的部分{3, 4, 5}。我们只要把{3, 4, 5}先颠倒过来变成{5, 4, 3}，然后再把之前的栈顶元素 2 放到最底部，也就变成了{5, 4, 3, 2}。

至于怎么把{3, 4, 5}颠倒过来……很多读者可能都想到这就是递归。也就是每一次试图颠倒一个栈的时候，现在栈顶元素 pop 出来，再颠倒剩下的元素组成的栈，最后把之前的栈顶元素放到剩下元素组成的栈的底部。递归结束的条件是剩下的栈已经空了。这种思路的代码如下：

```
// Reverse a stack recursively in three steps:
// 1. Pop the top element
// 2. Reverse the remaining stack
// 3. Add the top element to the bottom of the remaining stack
template<typename T> void ReverseStack(std::stack<T>& stack)
{
    if(!stack.empty())
    {
        T top = stack.top();
        stack.pop();
        ReverseStack(stack);
        AddToStackBottom(stack, top);
    }
}
```

我们需要考虑的另外一件事情是如何把一个元素 *e* 放到一个栈的底部，也就是如何实现 AddToStackBottom。这件事情不难，只需要把栈里原有的元素逐一 pop 出来。当栈为空的时候，push 元素 *e* 进栈，此时它就位于栈的底部了。然后再把栈里原有的元素按照 pop 相反的顺序逐一 push 进栈。

注意到我们在 push 元素 *e* 之前，我们已经把栈里原有的所有元素都 pop 出来了，我们需要把它们保存起来，以便之后能把他们再 push 回去。我们当然可以开辟一个数组来做，但这没有必要。由于我们可以用递归来做这件事情，而递归本身就是一个栈结构。我们可以用递归的栈来保存这些元素。

基于如上分析，我们可以写出 AddToStackBottom 的代码：

```
// Add an element to the bottom of a stack:
template<typename T> void AddToStackBottom(std::stack<T>& stack, T t)
{
    if(stack.empty())
    {
        stack.push(t);
    }
    else
    {
        T top = stack.top();
        stack.pop();
        AddToStackBottom(stack, t);
        stack.push(top);
    }
}
```

程序员面试题精选 100 题(40)-扑克牌的顺子

题目：从扑克牌中随机抽 5 张牌，判断是不是一个顺子，即这 5 张牌是不是连续的。2-10 为数字本身，A 为 1，J 为 11，Q 为 12，K 为 13，而大小王可以看成任意数字。

分析：这题目很有意思，是一个典型的寓教于乐的题目。

我们需要把扑克牌的背景抽象成计算机语言。不难想象，我们可以把 5 张牌看成由 5 个数字组成的数组。大小王是特殊的数字，我们不妨把它们都当成 0，这样和其他扑克牌代表的数字就不重复了。

接下来我们来分析怎样判断 5 个数字是不是连续的。最直观的是，我们把数组排序。但值得注意的是，由于 0 可以当成任意数字，我们可以用 0 去补满数组中的空缺。也就是排序之后的数组不是连续的，即相邻的两个数字相隔若干个数字，但如果我们有足够的 0 可以补满这两个数字的空缺，这个数组实际上还是连续的。举个例子，数组排序之后为{0，1，3，4，5}。在 1 和 3 之间空缺了一个 2，刚好我们有一个 0，也就是我们可以它当成 2 去填补这个空缺。

于是我们需要做三件事情：把数组排序，统计数组中 0 的个数，统计排序之后的数组相邻数字之间的空缺总数。如果空缺的总数小于或者等于 0 的个数，那么这个数组就是连续的；反之则不连续。最后，我们还需要注意的是，如果数组中的非 0 数字重复出现，则该数组不是连续的。换成扑克牌的描述方式，就是如果一副牌里含有对子，则不可能是顺子。

基于这个思路，我们可以写出如下的代码：

```
// Determine whether numbers in an array are continuous
// Parameters: numbers: an array, each number in the array is between
//             0 and maxNumber. 0 can be treeted as any number between
//             1 and maxNumber
//             maxNumber: the maximum number in the array numbers
bool IsContinuous(std::vector<int> numbers, int maxNumber)
{
    if(numbers.size() == 0 || maxNumber <=0)
        return false;

    // Sort the array numbers.
    std::sort(numbers.begin(), numbers.end());

    int numberOfZero = 0;
    int numberOfGap = 0;

    // how many 0s in the array?
    std::vector<int>::iterator smallerNumber = numbers.begin();
    while(smallerNumber != numbers.end() && *smallerNumber == 0)
    {
        numberOfZero++;
        ++smallerNumber;
    }

    // get the total gaps between all adjacent two numbers
    std::vector<int>::iterator biggerNumber = smallerNumber + 1;
    while(biggerNumber < numbers.end())
```

```
        {
            // if any non-zero number appears more than once in the array,
            // the array can't be continuous
            if(*biggerNumber == *smallerNumber)
                return false;

            numberOfGap += *biggerNumber - *smallerNumber - 1;
            smallerNumber = biggerNumber;
            ++biggerNumber;
        }

        return (numberOfGap > numberOfZero) ? false : true;
    }
```

本文为了让代码显得比较简洁，上述代码用 C++的标准模板库中的 vector 来表达数组，同时用函数 sort 排序。当然我们可以自己写排序算法。为了有更好的通用性，上述代码没有限定数组的长度和允许出现的最大数字。要解答原题，我们只需要确保传入的数组的长度是 5，并且 maxNumber 为 13 即可。